INITIATION PROGRAMMATION WEB

HTML 5 – CSS 3 – JavaScript

SUPPORT DE COURS

1re année de licence

Enseignants :

MM. AMEVOR, AGBOKA & ATTIOGBE

Année académique 2021-2022

OBJECTIFS DU COURS

- ✓ Connaître les standards du web et leurs spécificités
- ✓ Savoir créer des pages en HTML5
- ✓ Connaître les principales balises HTML5 existantes
- ✓ Créer des interfaces riches, interactives et ergonomiques
- ✓ Développer et débugger une application web
- ✓ Comprendre comment mettre en page et en forme à l'aide d'une feuille de styles CSS
- ✓ Construire une architecture technique conforme
- ✓ Apprendre les bases du JavaScript
- ✓ Dynamiser les pages avec du JavaScript
- ✓ Contrôler les données des formulaires

SOMMAIRE

OBJECTIFS DU COURS i
PARTIE I : LE LANGAGE HTML 5 : LES BALISES (Ossature/fond de la page) 1
I. Les balises et les attributs 3
II. Les différentes balises 10
PARTIE 2 : LANGAGE CSS 3 - LES STYLES (Ergonomie/forme de la page) 25
I. Les règles générales 25
II. Les sélecteurs 26
III. Les classes 28
IV. Sélecteur d'identifiant id 32
V. Les sélecteurs d'attributs 33
VI. Les sélecteurs contextuels parent-descendant 36
VII. Les sélecteurs d'éléments adjacents 38
VIII. La déclaration !important 39
IX. L'héritage 40
PARTIE 3 : BOOTSTRAP - Framework CSS 42
I. Présentation 42
II. Installation de bootstrap 46
III. Présentation de la grille 55
IV. Mise en page 75
V. Mise en forme des tableaux 78
VI. Mise en forme des formulaires 81
VII. Mise en forme de Menu (Horizontal et Vertical) 88
VIII. Les composants JavaScript du Bootstrap 89
PARTIE 4 : JAVASCRIPT – PROGRAMMATION (Dynamiser la page) 93
I. Historique du JavaScript 93
II. Les bases du langage JavaScript 93
III. Le DOM HTML (Document Object Model) 99
IV. Les boites de dialogues (alert, confirm, prompt) 106

V. Les structures conditionnelles/alternatives (IF..ELSE, SWITCH) 110
VI. Les structures itératives (WHILE, DO…WHILE, FOR, FOR…IN) 110
VII. Les tableaux 112
VIII. Les fonctions 113
IX. Les évènements 114
ANNEXE 118
I. La liste complète des balises HTML5 118
II. Liste des propriétes CSS3 et leurs significationss 122

PARTIE I : LE LANGAGE HTML 5 : LES BALISES (Ossature/fond de la page)

Généalogie de HTML 5

HTML 5 *(HyperText Markup Language)* est un langage de balisage (dit aussi langage de marquage) qui permet de structurer le contenu des pages web dans différents éléments.

Historiquement, les langages de balisage sont issus du langage SGML *(Standard Gene-ralized Markup Language)* créé en 1986 pour structurer des contenus très divers. Ce langage s'est révélé trop complexe pour être appliqué tel quel au Web, d'où la nécessité d'en créer une version allégée respectant les mêmes principes essentiels.

L'inventeur du HTML (1992), Tim Berners-Lee, l'avait conçu à l'origine comme un outil de structuration des contenus, principalement textuels, et non pas pour créer des présentations diversifiées. Ce sont les développements successifs, l'essor populaire du Web et la concurrence acharnée entre Netscape et Microsoft pour s'emparer du marché des navigateurs, qui ont détourné HTML de sa vocation première avec l'ajout d'éléments de présentation qui n'avaient rien à y faire. Voulant faire mieux que l'autre, chacun des deux grands a empilé des couches superflues sur HTML. Il faut également reconnaître que l'entrée du Web dans le grand public nécessitait de répondre à une demande d'interfaces graphiques plus esthétiques.

L'absence d'un langage particulier dédié uniquement à la présentation poussait effectivement les webmesters à utiliser tous les moyens pour créer des présentations visuelles agréables. L'apparition de CSS *(Cascading Styles Sheets),* créé en 1996 par Hâkon Wium Lie, aurait dû résoudre le problème du détournement de HTML de sa destination première. Mais les mauvaises habitudes étaient prises et la facilité faisait le reste.

L'apparition de HTML 4, et particulièrement de sa version *strict* associée à l'emploi systématique de CSS 2 (publié en 1998), pouvait apporter une solution efficace à ce problème. La création de XML *(eXtensible Markup Language)* en 1998 et son succès dans de multiples domaines d'application ont conduit le W3C à créer le langage XHTML, non plus comme une nouvelle version de HTML, mais comme une reformulation de HTML en tant qu'application XML. Au niveau des éléments et des attributs disponibles, il existait à vrai dire très peu de différences entre HTML 4 strict et XHTML 1.1.

L'impossibilité pour le W3C de trouver un consensus entre les éditeurs de navigateurs et les créateurs de moteurs de recherche pour faire évoluer XHTML a conduit un groupe indépendant, le WHATWG *(Web Hypertext Application Technology Working Group)* dirigé par Ian Hickson (aujourd'hui chez Google !), à entamer le développement de HTML 5

I. Les balises et les attributs

1. Les balises

Les pages HTML sont remplies de ce qu'on appelle des balises. Celles-ci sont invisibles à l'écran pour vos visiteurs, mais elles permettent à l'ordinateur de comprendre ce qu'il doit afficher. Les balises se repèrent facilement. Elles sont entourées de «chevrons», c'est-à-dire des symboles < et >, comme ceci : **<balise>**

Elles indiquent la nature du texte qu'elles encadrent. Elles peuvent signifier par exemple : « Ceci est le titre de la page », « Ceci est une image », « Ceci est un paragraphe de texte », etc.

On distingue deux types de balises : les balises en paires et les balises orphelines.

a. Les balises en paire

Elles s'ouvrent, contiennent du texte, et se ferment plus loin. Voici à quoi elles ressemblent : **<titre>**Ceci est un titre**</titre>**

On distingue une balise ouvrante (**<titre>**) et une balise fermante (**</titre>**) qui indique que le titre se termine. Cela signifie pour l'ordinateur que tout ce qui n'est pas entre ces deux balises n'est pas un titre.

b. Les balises orphelines

Ce sont des balises qui servent le plus souvent à insérer un élément à un endroit précis (par exemple une image). Il n'est pas nécessaire de délimiter le début et la fin de l'image, on veut juste dire à l'ordinateur « Insère une image ici ». Une balise orpheline s'écrit comme ceci : **<nom_balise />**

2. Les attributs

Les attributs sont un peu les options des balises. Ils viennent les compléter pour donner des informations supplémentaires. L'attribut se place après le nom de la balise ouvrante et a le plus souvent une valeur, comme ceci :

<nom_balise attribut="valeur"**>**

Exemple **:**

<nom_balise attribut1="Neil Armstrong" attribut2="21/07/1969"**>**

C'est un petit pas pour l'homme, mais un bond de géant pour l'humanité.

</nom_balise >

3. Structure de base d'une page HTML 5

```
<!doctype html>
<html>
	<head>
		<meta charset="utf-8" />
		<title>title</title>
		<link rel="shortcut icon" href="icon.ico" />
		<link rel="stylesheet" href="styles.css" />
		<script src="javascript.js"> </script>
	</head>
	<body>
		Contenu de la page
	</body>
</html>
```

a. Le doctype

<!DOCTYPE html>

La toute première ligne s'appelle le doctype. Elle est indispensable car c'est elle qui indique qu'il s'agit bien d'une page web HTML. Ce n'est pas vraiment une balise comme les autres (elle commence par un point d'exclamation), vous pouvez considérer que c'est un peu l'exception qui confirme la règle.

b. La balise </html>

<html></html>

C'est la balise principale du code. Elle englobe tout le contenu de votre page. Comme vous pouvez le voir, la balise fermante **</html>** se trouve tout à la fin du code.

c. L'en-tête <head> et e corps <body>

Une page web est constituée de deux parties :

L'en-tête **<head>** : cette section donne quelques informations générales sur la page comme son titre, l'encodage (pour la gestion des caractères spéciaux), etc. Cette section est généralement assez courte. Les informations que contient l'en-tête ne sont pas affichées sur la page, ce sont simplement des informations générales à destination de l'ordinateur. Elles sont cependant très importantes.

- <meta name="valeur" content="valeur">

L'attribut name définit ce que la valeur de content va décrire.

Exemples :

```
<meta charset="utf-8" />
<meta name="author" content="votre_nom">
<meta name="version" content="1.0">.
<meta name="description" content="description de la page">
<meta name="keywords" content="mot-cle1, mot-cle2, ...">
<meta name="generator" content="nom_du_generateur">
<meta name="robots" content="index,follow" />
<meta name="subject" content="sujet aborde">
```

- <link rel="name" href="lien_fichier" />

Il permet de faire des liens vers des fichiers ou pages

Exemples :

Une icône pour la page :

```
<link rel="shortcut icon" href="flavicon.ico" />
```

Une feuille de style pour la page

```
<link rel="stylesheet" href="styles.css" />
```

- <script src="fichier_script"> </script>

Elle permet d'utiliser un script (JavaScript par défaut) dans notre page. Cette balise peut être placée avant la fermeture de la balise <BODY> pour accélérer le chargement de la page.

Exemple :

```
<script src="javascript.js"> </script>
```

Le corps **<body>** : c'est là que se trouve la partie principale de la page. Tout ce que nous écrirons ici sera affiché à l'écran. C'est à l'intérieur du corps que nous écrirons la majeure partie de notre code. Pour le moment, le corps est vide.

Un **commentaire** en HTML est un texte qui sert simplement de mémo. Il n'est pas affiché, il n'est pas lu par l'ordinateur, cela ne change rien à l'affichage de la page. C'est une balise HTML avec une forme bien spéciale : *<!-- Inserrer un commentaire -->*

Exemple :

```
<!DOCTYPE html>
<html>
    <head>
     <!-- En-tête de la page -->
      <meta charset="utf-8" />
      <title>Titre</title>
   </head>
  <body>
  <!-- Corps de la page -->
 </body>
</html>
```

4. Les règles de base HTML 5

Un document HTML doit respecter certaines règles simples :

- Les éléments et les attributs sont insensibles à la casse. Par exemple, <body> et <B0DY> sont acceptés. Choisissez une casse selon votre goût et conservez-la.
- Les éléments non vides doivent avoir une balise d'ouverture et une balise de fermeture. Cela facilite la tâche des navigateurs. Par exemple, plutôt que d'écrire :

```
<ol>
<1i >élément 1 <1i> élément 2
on préférera le code suivant :
<ol>
    <1i> élément 1</li>
    <1i> élément 2</li>
</ol>
```

- Les éléments vides ne comportent qu'une seule balise et il est plus correct de les terminer par les caractères /> précédés d'un espace pour marquer la fin de l'élément. Par exemple, plutôt que d'écrire :

  <img src= "monimage.gif"> <hr>

  on préférera le code suivant :

   <hr />

- Les éléments ne doivent pas se chevaucher et donc obéir au principe premier ouvert, dernier fermé. Dans ce cas, le premier élément est le parent du second et celui-ci est enfant du premier. Par exemple, le code suivant est incorrect :

   Cette division contient un titre <hl> Important ! </hl>

  et doit être remplacé par :

   Cette division contient un titre <hl> Important ! </hl>

- Tous les attributs doivent avoir une valeur incluse entre guillemets ("). Les différents attributs du même élément doivent

être séparés par au moins un espace. Par exemple, plutôt que d'écrire :

 Texte important on préférera le code suivant :

 Texte important

- Une valeur doit être donnée à tous les attributs utilisés, y compris à ceux dont la valeur est unique. Par exemple, plutôt que d'écrire :

<input type= "checkbox" checked disabled /> on préférera le code suivant :

<input type= "checkbox" checked="checked" disabled="disabled" />

- Il y a des exceptions car certains attributs ne doivent pas avoir de valeur en particulier quand elle est booléenne. Dans ce cas, la présence seule du nom de l'attribut signifie implicitement la valeur true et l'absence de la valeur false.

5. L'environnement de travail

Pour créer des pages web avec HTML 5 et CSS 3, il faut être doté d'un environnement de travail adapté. En théorie, un simple éditeur de texte tel que le Bloc-notes de Windows ou Emacs sous Linux suffit. Cependant, comme la saisie des noms des différents éléments HTML 5 dans ce type d'éditeur peut devenir à la longue plutôt rébarbative, nous pouvons utiliser quelques outils susceptibles de faire gagner du temps : notepad++, sublime text, Dreamweaver etc.…

II. Les différentes balises

1. Balises relatives au paragraphe

  : pour organiser son texte en paragraphes ;

 : pour aller à la ligne.

2. Balises relatives au texte

<h1> </h1> : signifie « titre très important ». En général, on s'en sert pour afficher le titre de la page au début de celle-ci.

<h2> </h2> : signifie « titre important ».

<h3> </h3> : pareil, c'est un titre un peu moins important (on peut dire un « sous-titre » si vous voulez).

<h4> </h4> : titre encore moins important.

<h5> </h5> : titre pas important.

<h6> </h6> : titre vraiment, mais alors là vraiment pas important du tout.

  : permet de mettre le texte en italique

  : permet de dire que le texte est important (texte en gras)

<mark></mark> : permet de surligner le texte

3. Balises relatives à la liste

Les listes nous permettent souvent de mieux structurer notre texte et d'ordonner nos informations. Elles sont de plusieurs types :

- Les listes simples
- Les listes non ordonnées ou listes à puces ;
- Les listes ordonnées ou listes numérotées ou encore énumérations.

a. Les listes simples ou liste de définition

Elles sont définies par les balises suivantes :

```
<dl>
    <dt> </dt>
        <dd> </dd>
</dl>
```

Exemple :

```
<dl>
    <dt>HTML 5</dt>
    <dd> HTML (HyperText Markup Language) est un langage de balisage (dit aussi langage de marquage) qui permet de structurer le contenu des pages web dans différents éléments. </dd>
    <dt> SGML </dt>
    <dd langage SGML (Standard Gene-ralized Markup Language) créé en 1986 pour structurer des contenus très divers. Ce langage s'est révélé trop complexe pour être appliqué tel quel au Web, d'où la nécessité d'en créer une version allégée respectant les mêmes principes essentiels. </dd>
</dl>
```

b. Listes non ordonnées

Elle est illustrée par les balises suivantes :

 délimite toute la liste ;

 délimite un élément de la liste (une puce).

Exemple :

```
<ul Type=x>
    <li>Toto</li>
    <li>Tata</li>
    <li>Titi</li>
</ul>
```

X=circle; disc ou square

c. Listes ordonnées

 délimite toute la liste ;

 délimite un élément de la liste (une puce).

Exemple :

```
<h1>Liste des fournitures</h1>
<ol Type=y>
    <li>Cahier.</li>
    <li>Bic</li>
    <li>Ensemble géométrique</li>
</ol>
```

y=1; a ; A ; i ou I

4. Balise relative au lien

Pour indiquer un lien on utilise la balise suivante : <a href="chemin vers le fichier"></>

- Lien absolu

Exemple :

 Voulez-vous découvrir ce que c'est que la magie ?
Cliquer ici

- Lien relatif

Soit deux fichiers « index.html » et « apropos.html » se trouvant dans un même dossier. Si on se trouve sur le fichier « index.html », pour faire un lien vers « apropos.html » on écrira simplement <a href="apropos.html"></>

- Lien vers une ancre

Une ancre est un point de repère utilisé lorsque les pages html sont très longues. Il permet de faire un saut vers un autre titre plus bas de la page.

Exemple :

<h2 id="mon_ancre">Titre</h2>
Aller vers l'ancre
NB : On peut être amené à faire un lien vers une ancre qui se situe sur une autre page. Dans ce cas la déclaration se fera de la manière suivante :
Aller vers l'ancre sur une autre page

- Lien qui affiche une infobulle au survol

Tester voir

- Un lien qui ouvre une nouvelle fenêtre

Pour le faire, il faut rajouter l'attribut target="_blank" à la balise <a>

Nouvelle fenêtre

- Lien pour envoyer un e-mail

Envoyez-moi un e-mail

- Lien pour télécharger un fichier

Télécharger le fichier

Exercice de synthèse 1

Vous allez créer une page, appelée Puces.html, qui fera apparaître plusieurs mises en forme de listes. La page comprendra 2 parties. Il est conseillé d'aller vérifier dans le navigateur votre travail entre chaque partie. Pour cela, enregistrez votre fichier et réactualiser la page dans le navigateur.

Description de la page à créer :

- Le fond de la page sera de la couleur "palegoldenrod" et le texte sera "marroon".

- Le titre de la page sera : Les listes

1ère partie :

- un titre de niveau 1, centré : LISTES A PUCES
- une liste à puces de trois choix appelés : choix1, choix2, choix3.
- Une ligne séparatrice

2ème partie :

- un titre de niveau 1, centré : LISTES NUMEROTEES
- Une liste numérotée de deux choix appelés : choix1, choix2.

5. Balise relative aux images

On insère une image à travers la balise orpheline et deux attributs

On distingue différents formats d'image dont les plus utilisés sont :

JPEG : pour les photos,

PNG : pour toutes les autres illustrations,

GIF : similaire au PNG, plus limité en nombre de couleurs mais qui peut être animé.

Exemple :

<img src="ma_photo.jpg" alt="Photo préférée" **/>**

Pour ajouter une infobulle à une image il suffit d'ajouter l'attribut ' title ' à la balise <img/>

Exemple :

<img src=" ma_photo.jpg " alt=" Photo préférée " title="Au survol"**/>**

6. Balise relatives aux audios et aux vidéos

On insère les audios et les vidéos grâce aux balises suivantes :

<audio src="URL fichier" ></audio>

<video src="URL fichier">...</video>

Voici les attributs qu'on peut ajouter aux balises <audio> et <video>

- controls="controls" qui permet d'afficher des éléments de contrôle (lecture, pause, retour, durée...) en dessous de la vidéo.
- autoplay="autoplay" qui déclenche la lecture dès l'ouverture de la page et le début du chargement du fichier. Pour obtenir le cas contraire, il faut simplement supprimer cet attribut.
- poster="URL image" qui affiche une image fixe le temps que la vidéo se charge et démarre soit automatiquement, soit par action du visiteur.
- 1oop="1 oop" pour lire le fiehier en boucle (c'est le cas sur les sites pour les publicités par exemple) ; sinon, il faut enlever cet attribut.
- muted="muted" pour supprimer le son (non reconnu à ce jour).
- preload="none | metadata | auto" qui détermine le comportement du navigateur quant au préchargement du fichier. Avec la valeur none, aucun préchargement n'est effectué jusqu'à ce que l'utilisateur ne démarre la lecture. Avec la valeur metadata, seules les métadonnées sont préchargées. La valeur auto entraîne le préchargement du fichier en même temps que celui de la page.

- height et width pour imposer des dimensions au cadre de la vidéo. Si une seule valeur est donnée, l'autre est déterminée pour conserver les proportions de l'image.

Exemple :

```
<!DOCTYPE html>
<html>
<head>
    <title>Une vidéo en HTML 5</title>
    <meta charset="UTF-8"/>
</head>
<body>
    <video src="shakira.mp4" type="video/mp4" controls = "controls" height="300" width="450" autoplay="autoplay" oop="loop">
    </video>
</body>
</html>
```

7. Balise relative aux figures

En HTML 5, on dispose de la balise <figure></figure> qui permet d'insérer les figures.

Exemple :

```
<figure>
    <img src="images/blocnotes.png" alt="Bloc-Notes" />
</figure>
```

On peut aussi ajouter une légende à une figure grâce à la balise <figcaption></figcaption>

```
<figure>
    <img src="images/blocnotes.png" alt="Bloc-Notes" />
    <figcaption>Le logiciel Bloc-Notes</figcaption>
</figure>
```

8. Les balises relatives aux tableaux

Un tableau est défini par les balises suivantes :

```
<table/>
    <tr>
        <td>
        </td>
    </tr>
<table/>
```

Exemple 1:

```
<Table border=2 width="40%">
    <tr>
    <td colspan="4" align="center"> 1</td>
    </tr>
    <tr>
        <td> A </td>
        <td> B </td>
        <td> C </td>
        <td> D </td>
    </tr>
    <tr>
        <td> E </td>
        <td> F </td>
```

```
            <td> G </td>
            <td> H </td>
        </tr>
        <tr>
            <td> I </td>
            <td> J </td>
            <td> K </td>
            <td> L </td>
        </tr>
</Table>
```

Exemple 2 : Réaliser le tableau suivant

<table>
<tr><td rowspan="4">1</td><td colspan="4">2</td></tr>
<tr><td rowspan="2" colspan="2">3</td><td>4</td><td>5</td></tr>
<tr><td>6</td><td>7</td></tr>
<tr><td>8</td><td colspan="3">9</td></tr>
<tr><td>10</td><td>11</td><td>12</td><td>13</td></tr>
</table>

Exercice de synthèse 2

Vous allez créer une page, appelée Tableaux.html, qui fera apparaître plusieurs tableaux. La page comprendra 2 parties. Il est conseillé d'aller vérifier dans le navigateur votre travail entre chaque partie. Pour cela, enregistrez votre fichier et réactualiser la page dans le navigateur.

Description de la page à créer :

- Le fond de la page sera de la couleur "palegoldenrod" et le texte sera "marroon".
- Le titre de la page sera : LES TABLEAUX

1ère partie :

- un titre de niveau 1, centré : TABLEAU SIMPLE
- Vous réaliserez le tableau dont la description en HTML est donnée ci-dessus. Ce tableau sera centré.
- Une ligne séparatrice

2ème partie :

- un titre de niveau 1, centré : TABLEAU PLUS COMPLEXES
- Vous réaliserez un tableau, centré, similaire au tableau ci-dessous :

Cellule 1	Cellule 2	Cellule 3	Cellule 4
Cellule fusionnée sur 4 colonnes			
Cellule 6	Cellule 7	Cellule 8	Cellule 9

- Une ligne séparatrice de 200 pixels de large
- Vous réaliserez un tableau, centré, similaire au tableau ci-dessous :

Cellule 1	Cellule 2	Fusion de 2 lignes
Cellule 3	Cellule 4	

- Une ligne séparatrice de 200 pixels de large
- Vous réaliserez un tableau, centré, similaire au tableau ci-dessous :

<table>
<tr><th colspan="3">TITRE 1</th></tr>
<tr><td colspan="2">Une petite image choisie sur le site de Stendhal</td><td>Cellule 1</td></tr>
<tr><th colspan="3">TITRE 2</th></tr>
<tr><td>Cellule 2</td><td rowspan="2">Une petite image choisie sur le site de Stendhal</td><td>Cellule 4</td></tr>
<tr><td>Cellule 3</td><td>Cellule 5</td></tr>
</table>

9. Balises relatives au formulaire

La structure générale d'un formulaire est la suivante :

```
<form method=”#” action="#">
    <fieldset>
      <1egend>Formulaire type</legend>
       <hl>Les composants du formulaire</hl>
       <!-- Composants du formulaire -->
    </fieldset>
</form>
```

Les composants d'un formulaire :

1. Zone de texte

```
<input type=”#” name="#" value="#">
```

2. L'élément textarea

```
<Textarea name="commentaire" rows=10 cols=40>
   Tapez vos commentaires ici
</textarea>
```

3. L'élément radio

Exemple:

<input type="radio" name="sexe" value="F" checked> Feminin
<input type="radio" name="sexe" value="M"> Masculin

4. Case à cocher

Exemple:

<input type="checkbox" name="ch" value="Word" checked> Word
<input type="checkbox" name="ch" value="Excel"> Excel
<input type="checkbox" name="ch" value="Access"> Access

5. Menu déroulante

<select name="#">
<option><option>
</select >

Example**:**

<select name="liste">
<option> Taro
<option> Patate
<option> Igname
<option selected> Pomme de Terre
<option> Oignon
<option> Ail
</select >

<select name="liste1" size=4>
<option> Taro
<option> Patate

```
        <option> Igname
        <option selected> Pomme de Terre
        <option> Oignon
        <option> Ail
</select >

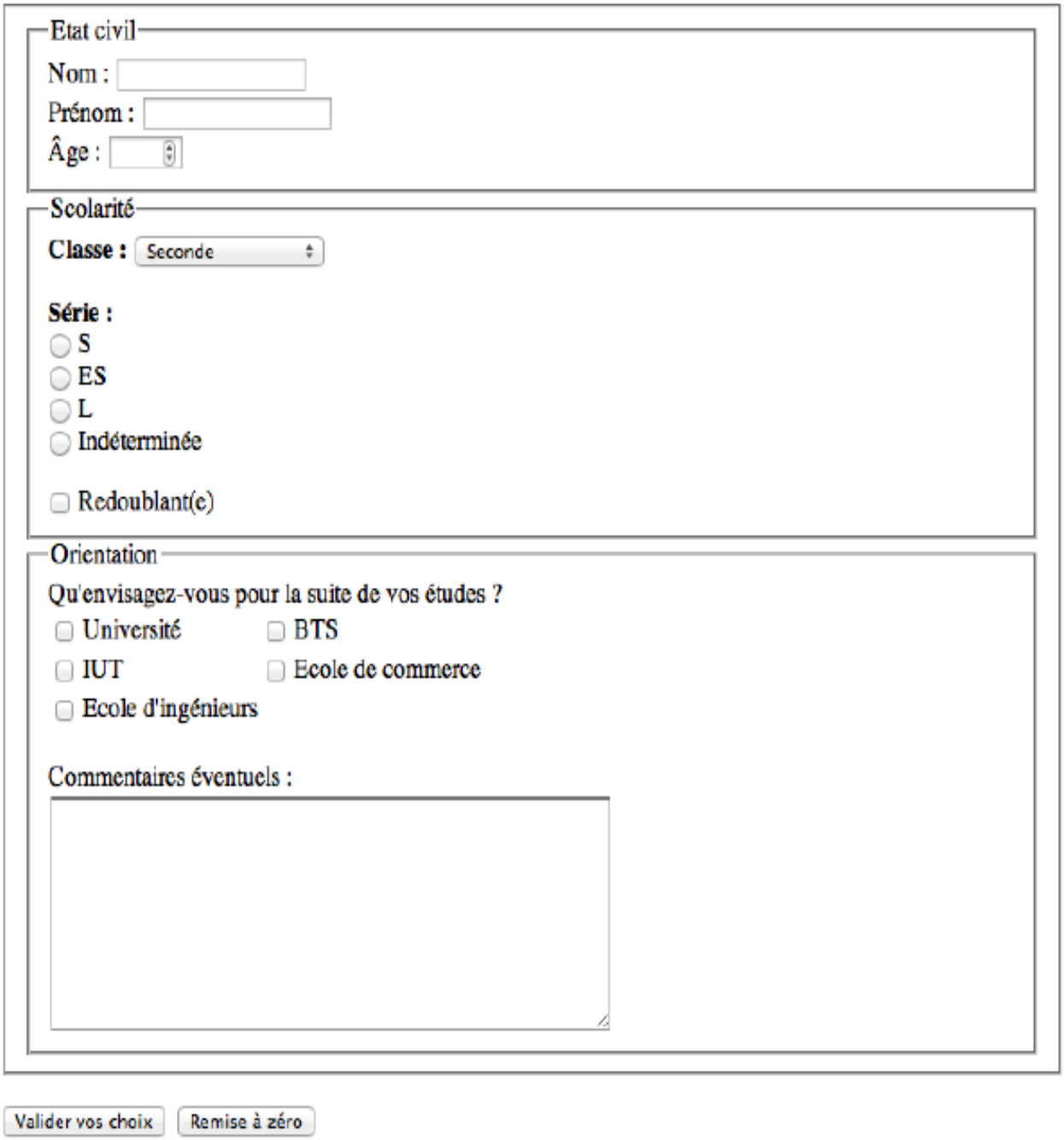
<select name="liste2" size=4 multiple>
        <option> Taro
        <option selected > Patate
        <option> Igname
        <option selected> Pomme de Terre
        <option> Oignon
        <option> Ail
</select >
```

6. Les boutons

```
<input type="submit" name="bouton1" value="Envoyer">
<input type="reset" name="bouton2" value="Effacer">
```

Exercice de synthèse 3

Ecrire le code HTML pour obtenir un formulaire ressemblant à la capture d'écran ci-dessous.


Etat civil
Nom :
Prénom :
Âge :
Scolarité
Classe : Seconde
Série :
S
ES
L
Indéterminée
Redoublant(e)
Orientation
Qu'envisagez-vous pour la suite de vos études ?
Université
BTS
IUT
Ecole de commerce
Ecole d'ingénieurs
Commentaires éventuels :
Valider vos choix
Remise à zéro

PARTIE 2 : LANGAGE CSS 3 - LES STYLES (Ergonomie/forme de la page)

Les feuilles de style CSS permettent de mettre en forme les éléments du langage HTML.

I. Les règles générales

Une déclaration de style comporte plusieurs parties, selon l'ordre suivant.

- ✓ Un sélecteur qui va déterminer à quel élément et éventuellement dans quelles conditions va s'appliquer le style. Autant que les propriétés, c'est la variété des sélecteurs qui fait la richesse de CSS.

- ✓ La déclaration des propriétés que l'on veut voir appliquées à l'élément sélectionné. Elle doit être incluse entre des accolades ouvrante ({) et fermante (}).

- ✓ Dans ces accolades doivent apparaître une ou plusieurs propriétés, déterminées chacune par un mot-clé propre à CSS suivi du caractère deux-points (:), puis de la valeur attribuée à cette propriété. Si nous définissons plusieurs propriétés dans le même style, il faut séparer chaque déclaration de la précédente par le caractère point-virgule (;)

- ✓ Les propriétés sont en nombre limité et font l'objet d'une recommandation du W3C. La version actuelle de CSS est la version 3, dans laquelle ont été éliminées un certain nombre de propriétés qui n'étaient mises en application par aucun navigateur, et qui comporte de nouvelles propriétés par rapport à la version CSS 2.

Syntaxe :

```
Sélecteur {
 Propriéte 1 : valeur 1 ;
Propriéte 2 : valeur  2 ;
}
```

Exemple

```
div {color : red ; background-color :yellow ;}
```

II. Les sélecteurs

Une des grandes richesses de CSS est la multiplicité des possibilités de sélection des éléments auxquels on veut attribuer un style donné. Cette très grande diversité permet en effet d'appliquer un style aussi facilement à tous les éléments, en une seule ligne de code, qu'à un unique élément isolé dans la page web, sans avoir à écrire la définition localement. De plus, la combinaison de plusieurs sélecteurs dans la même déclaration ouvre la voie à une quasi-infinité de solutions, propres à répondre à tous les besoins, même les plus complexes.

1. Sélectionner un seul élément

Il s'agit de la sélection la plus simple, puisque le sélecteur est constitué du nom de l'élément sans les caractères de début < et de fin de balise />. Nous écrivons par exemple :

p {color : yellow ; background-color : blue;} pour que le texte de tous les paragraphes figure en jaune sur fond bleu.

2. Sélectionner plusieurs éléments

Nous pouvons très facilement appliquer le même style à plusieurs éléments différents en les énumérant et en les séparant par une virgule dans le sélecteur.

Plutôt que de multiplier les définitions :

hl {color : black ; background-color : red;}

div {color : black ; background-color : red;}

p {color : black ; background-color : red;}

Nous pouvons écrire le style suivant :

hl,div,p {color : black ; background-color : red;}

Cette possibilité de regroupement peut être utile pour définir des styles communs à un ensemble d'éléments en écrivant ce type de sélecteur pour cet ensemble, puis en ajoutant d'autres propriétés spécifiques à un des éléments de la liste. On définit ainsi une sorte de tronc commun à un groupe, puis on affine chacun de ses composants.

Si nous écrivons par exemple le code suivant :

hl,div,p {color : black ; background-color : red;}

div {margin : 20px;}

3. Le sélecteur universel

Pour appliquer un style à tous les éléments, nous utiliserons le sélecteur universel ***** avant la définition d'une ou plusieurs propriétés. Par exemple, pour que la couleur de fond de tous les éléments soit le jaune, nous écrirons :

```
*{background-color : yellow;}
```

Cela n'empêche pas de modifier cette couleur de fond pour un élément particulier, en la redéfinissant uniquement pour celui-ci, par exemple :

```
*{background-color : yellow;}
p{background-color : gray;}
```

Dans ce cas, tous les éléments ont un fond jaune, sauf <p> qui a un fond gris redéfini spécialement.

III. Les classes

1. Appliquer une classe à un élément

Nous avons vu que tous les éléments HTML 5 possèdent l'attribut class. Ce dernier permet d'appliquer un style défini dans une classe à un élément dont l'attribut class se voit attribuer le nom de cette classe. Pour créer une classe, le sélecteur est constitué du nom choisi pour la classe précédé d'un point (.). Le nom de la classe peut être un mot quelconque, en évitant quand même les noms des

propriétés CSS et des éléments HTML 5 car cela occasionnerait des confusions. Nous pouvons par exemple définir la classe nommée evidence en écrivant le code :

.evidence (color : red;}

À ce stade, la classe est abstraite et ne s'applique à aucun élément. Pour mettre en évidence un paragraphe précis de la page avec un texte rouge, nous devons alors écrire dans le code HTML 5 :

Texte contenu du paragraphe

Les classes présentent l'intérêt de pouvoir s'appliquer à n'importe quel élément, n'importe où dans le code de la page. Notre classe evidence peut donc s'appliquer à un titre <hl>, une division ou un élément , simplement en écrivant pour chacun d'entre eux l'attribut class="evidence".

Nous pouvons également définir une classe en la déclarant applicable seulement à un élément en faisant précéder son nom de celui de l'élément. Nous pouvons écrire par exemple :

div.jaune {color : yellow;}

Dans ce cas, seules les divisions ayant un attribut class dont la valeur est jaune ont un texte jaune. Les autres éléments, et même s'ils ont le même attribut avec la même valeur, n'ont pas de style défini dans cette classe.

Le sélecteur universel * peut également être employé à la place du nom d'un élément dans la définition d'une classe. Le style

s'applique alors à tous les éléments dont l'attribut class a pour valeur le nom de la classe. Nous écrivons alors par exemple :

*.jaune {color : yellow;}

Il est possible de définir d'abord une classe abstraite, puis de la particulariser en ajoutant une autre propriété pour un élément qui utilisera la même classe. Dans le code CSS ci-après :

.rouge {color : red;}
div.rouge {background-color : blue;}

Texte contenu de la division
Implique que le texte contenu dans l'élément est affiché en rouge sur fond bleu.

2. Appliquer plusieurs classes au même élément

L'avantage de définir des classes abstraites est, nous l'avons vu, qu'elles peuvent s'appliquer à n'importe quel élément. Leur puissance peut être multipliée car nous pouvons appliquer plusieurs classes indépendantes à un même élément. Celui-ci a alors la combinaison des propriétés de chacune des classes. Pour utiliser plusieurs classes dans le même élément HTML 5, il faut donner à son attribut class la liste des noms des classes en les séparant par un espace comme ceci :
 Ceci est un texte avec la classe 1 et 2

Les combinaisons d'emploi des classes sont alors multiples, chaque classe pouvant définir une caractéristique, et chaque élément pouvant en utiliser plusieurs au choix.
Exemple:

```
<!DOCTYPE html>
<html>
  <head>
   <meta http-equiv="Content-type" content="text/html; charset=UTF-8" />
   <title> Les classes de style </title>
        <style type="text/css" title="classes">
        *.jaune{color: green;}
         div.jaune {color: yellow;}
        .classel {color: red;}
        .classe2 {font-style: italic}
        .classe3 {background-color: #CCC;}
        </sty1e>
  </head>
<body>
<hl class="jaune">HTML 5 et CSS</hl>
<div class="classel">
Ceci est un texte avec la classe1 (texte rouge) <br/> <br/>
</div>
<div class="classel classe2">
```

Ceci est un texte avec la classe 1 et 2 (texte rouge et en italique)

@
Ceci est un texte avec les classes 1 et 3 (texte rouge et fond gris)

Ceci est un texte avec les classes div.jaune, 2 et 3 (texte jaune, en italique et fond gris)

</body>
</html>

IV. Sélecteur d'identifiant id

Pratiquement, chaque élément peut avoir un attribut id qui doit être unique dans une page donnée. Nous pouvons écrire un style qui ne sera applicable qu'à l'élément dont l'attribut id a une valeur précise en donnant cette valeur au sélecteur (comme pour une classe) et en le faisant précéder du caractère dièse (#).
En écrivant le style suivant :

```
div {color: black;}
#bleu {color: white; background-color : blue;}

<div id="bleu">Texte en blanc sur bleu</div>
<div>Texte en noir </div>
```

Seul l'élément contenant l'attribut id="bleu" bénéficie du texte en blanc sur fond bleu, et aucun autre, car l'identifiant doit être unique dans une page, et les autres éléments ont un texte noir. Ce type de définition est très ciblé car le style ne peut s'appliquer qu'à un seul élément du fait de l'unicité de l'identifiant id.

NB : Les valeurs de l'attribut id sont sensibles à la casse en HTML 5 ; bleu et B1eu sont donc deux identifiants différents (sauf dans Internet Explorer pour qui la casse importe peu).

V. Les sélecteurs d'attributs

Il est également possible d'appliquer un style à un élément déterminé dès qu'il possède un attribut donné, quelle que soit la valeur de cet attribut. Pour appliquer ce sélecteur, le nom de l'élément doit être suivi du nom de l'attribut placé entre crochets ([) et (]). En définissant le style suivant :

<abbr>[title] {color: red; background-color: gray;}

Tous les éléments <abbr> qui possèdent un attribut title, quelle que soit sa valeur, ont un contenu affiché en rouge sur fond gris. De même, en définissant le style pour l'élément <img/> :

img[ismap] {border-color : red; border-weight: 2px;}

Toutes les images ayant un attribut ismap ont une bordure rouge de deux pixels de large.
Nous pouvons également créer un style applicable à tous les éléments qui possèdent un attribut donné en utilisant le sélecteur universel * placé devant les crochets qui contiennent le nom de l'attribut choisi. Si nous définissons le style suivant :

*[title] {background-color : yellow;}, ce sont tous les éléments ayant l'attribut title qui sont affichés avec un fond jaune.
Comme pour les classes, il est possible de sélectionner plusieurs attributs pour un élément en faisant suivre son nom de plusieurs attributs entre crochets. Si nous écrivons le style suivant :
h2[title][id]{background-color: yel1ow;}, seuls les titres <h2> ayant à la fois les attributs title et id ont une couleur de fond jaune.

1. Les sélecteurs de valeur d'attribut

Nous pouvons appliquer un style à un élément à condition que ses attributs aient une valeur précise en utilisant la syntaxe suivante :
element [attribut="valeur"] {Définition du style;}

Exemple :
code[title="code JavaScript"] {color: blue;}
Il est aussi possible de particulariser davantage l'application du style en sélectionnant plusieurs attributs et leurs valeurs en utilisant la syntaxe :
element[attributl="valeurl"][attribut2="valeur2"] {Définition du style;}

Par exemple, avec la définition suivante, appliquée à une cellule de tableau :

```
td {font-size: 12px;}
td [title="nom"] [align="center"] {font-size: 14px; color: red;}
```

seules les cellules ayant un attribut title avec la valeur nom et un attribut align ayant la valeur center sont affichées avec un texte rouge et avec une taille de fonte de 14 pixels, alors que le texte des autres cellules a une taille de 12 pixels.

2. Les sélecteurs de valeur d'attribut CSS 3

De nouveaux sélecteurs ont été créés dans CSS 3. Ils permettent d'affecter un style à un élément de manière encore plus fine que les précédents, ou redéfinissent la sélection d'une valeur d'attribut. Les syntaxes à utiliser sont les suivantes :

```
element[attribut ^= "valeur"]{définition du style}
element[attribut $="valeur"]{définition du style}
element[attribut *="valeur"] {définition du style}
```

Dans le premier cas, le style est attribué à l'élément dont l'attribut commence par la chaîne valeur, dans le deuxième cas si l'attribut finit par cette même chaîne, et enfin dans le troisième, s'il contient simplement cette chaîne à n'importe quelle place.

En écrivant par exemple les styles suivants :

```
td[id ^= ”nom"]{background-color:blue;color:white;}
```

td[id $= "nom"]{background-color:red;color:white;}

td[id *= nnom"]{background-color:black;color:white;}

Puis le code HTML suivant :

<td id="nom de famille">Engels</td>

<td id="mon nom">Engels</td>

<td id="c mon nom de famille">Engels</td>

VI. Les sélecteurs contextuels parent-descendant

Plutôt que de définir un style pour toutes les occurrences d'un élément, nous pouvons souhaiter ne l'appliquer qu'en fonction de sa position relative par rapport à un autre dans la hiérarchie des éléments de la page. Ce type de sélecteur est dit contextuel. Nous pouvons, par exemple, définir un style général pour l'élément et vouloir lui en appliquer un autre quand il se trouve inclus dans un élément . Pour cela, il faut utiliser la syntaxe suivante :

element_parent element_enfant {Définition des styles;}

En écrivant par exemple le style suivant :

p (color: blue;}

div p {color: red;}

Et en l'appliquant au code HTML 5 ci-après :

Texte du paragraphe inclus dans div

Texte d'un paragraphe non inclus dans div

Seuls les contenus des éléments <p> inclus dans sont de couleur rouge, tous les autres étant bleus, et le texte inclus directement dans a la couleur par défaut qui est le noir.
Il est aussi possible de préciser la hiérarchie en appliquant plus de deux éléments en les séparant tous par un espace.
Pour appliquer un style différent aux éléments de liste <li> inclus directement dans une liste non ordonnée <ul> ou dans une liste ordonnée <ol> elle-même incluse dans une liste non ordonnée <ul>, nous pouvons définir les sélecteurs suivants :

ul li (background-color : #EEE; color: black;}
ul li ol li (background-color : gray; color: w hite;}

Une fois qu'ils sont appliqués au code des listes ci-après :

```
<ul>
  <1i> Item de niveau 1
   <ol>
        <li> Item de niveau 2 A </li>
        <li> Item de niveau 2 B</1i>
        <li> Item de niveau 2 C</1i>
   </ol>
  </1i>
</ul>
```

Nous obtenons pour les items de niveau 1 un texte noir sur fond gris, et pour les trois items de niveau 2, un texte blanc sur fond gris.

VII. Les sélecteurs d'éléments adjacents

Les sélecteurs précédents font appel à la notion de descendance parent-enfant. Si on considère dans le code de la page deux éléments consécutifs enfants du même parent, comme deux éléments de bloc inclus dans <body> par exemple, il n'existe pas de relation de descendance entre eux mais une relation de fratrie. Pour sélectionner ce type de relation entre éléments, nous disposons du sélecteur + qu'il faut utiliser en adoptant la syntaxe suivante :

elementl+element2 {Définitions des styles;}

Dans ce cas, le style s'appliquera à l'élément de type 2 uniquement s'il suit immédiatement un élément de type 1 dans le code HTML 5 sans y être inclus. Si nous définissons les styles suivants :

```
p {background-color: yellow; color: blue;}
div+p {background-color: blue; color: yellow;}
```

puis le code HTML 5 ci-après :

```
<div>Les standards HTML 5 et CSS 3 s'imposent aujourd'hui
    <p>Texte d'un paragraphe enfant de div</p>
</di v>
<p>Texte d'un paragraphe frère de div</p>
<p>Texte d'un paragraphe frère de div</p >
```

Seul le deuxième élément <p> a le style texte jaune sur fond bleu, le premier n'étant pas frère mais enfant de l'élément . Le troisième paragraphe aura de plus le même style que le premier car il ne suit pas immédiatement l'élément .
On note que si les deux éléments sont reliés par le sélecteur +, seul le second présente un style défini. En effet, si nous écrivons le code suivant au lieu des styles précédents :

```
p {background-color: yellow; color: blue;}
p+p {background-color: blue; color: yellow;}
```

et si nous l'appliquons au même code HTML 5 que précédemment, seul le troisième paragraphe a le style texte jaune sur fond bleu. Les premier et deuxième paragraphes bien qu'étant consécutifs n'ont pas le même parent direct. Notons encore qu'avec ce dernier style, si un élément contenait plus de deux éléments <p>, le style jaune sur bleu créé avec le sélecteur p+p s'appliquerait à tous les paragraphes enfants de sauf le premier. De même, s'il existait deux paragraphes consécutifs en dehors de l'élément , le second bénéficierait également du même style.

VIII. La déclaration !important

Chaque déclaration de style peut revêtir un caractère de plus grande importance par rapport à une autre déclaration concernant le même élément et la même propriété qui comporte une valeur différente. Ces deux déclarations peuvent entrer en conflit au moment de la création de la présentation par le navigateur. Pour

donner cette importance à un style, il faut insérer la déclaration d'importance à l'aide du mot-clé {important en le plaçant entre la valeur attribuée à la propriété et le point-virgule qui termine la déclaration. Dans l'exemple suivant :

```
*{color: black !important; background-color : yellow;}
div{color: blue; background-color: white;}
```

Les couleurs de texte et de fond des sélecteurs * et div sont en conflit, mais comme la propriété color définie dans le sélecteur universel * est marquée !important, le texte de la division figure en noir. En revanche, le fond de la division est de couleur blanche car la valeur yellow n'est pas marquée !important et que la déclaration effectuée dans div est spécifique.

Il est évidemment possible d'utiliser la déclaration !important pour plusieurs propriétés dans la même déclaration, par exemple :

```
*{color: black !important; background-color: yellow !important;}
div{color: blue; background-color: white;}
```

Dans ce cas, tous les éléments ont un contenu affiché en noir sur fond jaune, et tous les styles définis en propre pour cet élément sont ignorés.

IX. L'héritage

L'héritage est le fait qu'un élément enfant possède les mêmes styles que l'élément qui le contient (son parent dans la hiérarchie des

éléments d'une page). Nous pouvons par exemple définir les styles suivants :

Div{color: white; background-color: blue;}

Si dans le code HTML 5 de la page figurent les éléments suivants :
Texte
Premier paragraphe
<span class=H">HTML 5 et les stylesCSS 2.1 sont indispensables
 à tous !

L'élément est enfant de et les éléments et sont eux-mêmes enfants de . Par héritage et bien sûr faute d'avoir définis des styles propres pour les éléments , et , ceux-ci ont un contenu qui possède les styles définis pour leur parent direct ou indirect . Ils sont donc tous en blanc sur fond bleu. Si nous créons un style différent pour l'élément , ses éléments enfants héritent alors de ces styles et non plus de ceux de l'élément .

De même, si nous définissons un style pour les éléments de listes ordonnées ou non <ol> et <ul>, tous les items de la liste, quel que soit leur niveau d'imbrication, ont les mêmes caractéristiques par défaut sans qu'il faille créer un style propre pour eux.

PARTIE 3 : BOOTSTRAP - Framework CSS

I. Présentation

1. Bibliothèque

Il existe des bibliothèques côté serveur (désignés backend en anglais), et d'autres côté client (désignés frontend en anglais). Bootstrap fait partie de cette deuxième catégorie. Les bibliothèques CSS sont spécialisées, comme leur nom l'indique, dans les CSS ! C'est-à-dire qu'ils nous aident à mettre en forme les pages web : organisation, aspect, animation... Elles sont à la mode et il en existe un certain nombre, en voici quelques-uns :

- Knacss
- Blueprint
- Unsemantic
- YUI
- 52Framework
- Pure
- Materializecss
- Semantic-ui
- Getuikit
- etc.

Bootstrap est une bibliothèque CSS, mais pas seulement, puisqu'il embarque également des composants HTML et JavaScript. Il comporte un système de grille simple et efficace pour mettre en

ordre l'aspect visuel d'une page web. Il apporte du style pour les boutons, les formulaires, la navigation…

Il permet ainsi de concevoir un site web rapidement et avec peu de lignes de code ajoutées.

La bibliothèque la plus proche de Bootstrap est sans doute Foundation qui est présenté comme « The most advanced responsive front-end framework in the world ». Cette absence de modestie est-elle de mise ? Je pense que c'est surtout une affaire de goût et de préférence personnelle. En tout cas en termes de popularité c'est Bootstrap qui l'emporte haut la main.

Les intérêts d'utiliser une bibliothèque

- Les navigateurs sont pleins de fantaisies et ont des comportements très différents malgré leur lente convergence vers les standards. Les bibliothèques sont cross-browser, c'est-à-dire que la présentation est similaire, quel que soit le navigateur utilisé et d'une parfaite compatibilité.
- Les bibliothèques CSS font gagner du temps de développement parce qu'ils nous proposent les fondations de la présentation.
- Les bibliothèques CSS normalisent la présentation en proposant un ensemble homogène de styles.
- Les bibliothèques CSS proposent en général une grille pour faciliter le positionnement des éléments.
- Les bibliothèques CSS offrent souvent des éléments complémentaires : boutons esthétiques, barres de navigation, etc.

- La grande diffusion de nouveaux moyens de visualisation du web (smartphones, tablettes…) impose de s'adapter aux tailles d'écran très variées, les bibliothèques CSS prennent généralement en compte cette contrainte.

Les inconvénients d'utiliser une bibliothèque

Pour utiliser efficacement une bibliothèque, il faut bien la connaître, ce qui implique un temps d'apprentissage. La normalisation de la présentation peut devenir lassante en lissant les effets visuels. Par rapport aux deux inconvénients énoncés, Bootstrap est d'une prise en main rapide même pour un débutant et est totalement configurable parce qu'il est construit avec Sass.

2. Découverte de bootstrap

Historique de bootstrap

Vous connaissez forcément Twitter, un des principaux réseaux sociaux qui inondent la planète de liens virtuels entre les humains devenus des noyaux cybernétiques. Le projet Bootstrap a été créé au départ par Mark Otto et Jacob Thornton pour répondre à des besoins internes de développement de cette entreprise au niveau de l'uniformisation de l'aspect des pages web. Il s'agissait juste de stylisation CSS, mais la bibliothèque s'est ensuite enrichie de composants JavaScript.

Il a ensuite été publié en 2011 en devenant rapidement populaire parce qu'il est venu se positionner dans un espacevacant du développement. Son système de grille de 12 colonnes est devenu une référence. D'autre part sa mise en œuvre est aisée comme nous

allons bientôt le voir. Il a été mis à disposition du public sous licence MIT. La bibliothèque en est actuellement à la version 5.

Contenu de la bibliothèque

Bootstrap propose :

- une mise en page basée sur une grille de 12 colonnes fondée sur flexbox ;
- l'utilisation de Normalize auquel est ajoutée l'extension Reboot dans la version 4 ;
- du code fondé sur HTML5 et CSS3 ;
- une bibliothèque totalement open source sous licence MIT ;
- du code qui tient compte du format d'affichage des principaux outils de navigation (responsive design) : smartphones, tablettes… ;
- des plugins jQuery de qualité ;
- un résultat cross-browser (la prise en charge de IE8 a été abandonnée avec la version 4), donc une garantie de compatibilité maximale ;
- une bonne documentation ;
- la garantie d'une évolution permanente ;
- une mine de ressources variées sur le web ;
- une architecture basée sur Sass, un outil bien pratique qui étend les possibilités de CSS.

II. Installation de bootstrap

1. Les fichiers de bootstrap

Pour récupérer les fichiers de Bootstrap c'est simple : cliquez sur le bouton de téléchargement (download) sur le site : http://getbootstrap.com.

Les fichiers utiles pour simplement utiliser Bootstrap se situent dans le dossier dist (« distribution »).

Voyons un peu les fichiers les plus importants :

- **bootstrap.css** comporte les classes de base de Bootstrap ;
- **bootstrap.js** contient le code JavaScript des composants de Bootstrap ;
- **bootstrap.bundle.js** contient le code JavaScript des composants de Bootstrap comme bootstrap.js, mais en plus la bibliothèque popper.js ;

Les fichiers « min » (**bootstrap.min.css, bootstrap.bundle.min.js et bootstrap.min.js**) contiennent le même code que leurs équivalents (bootstrap.css, bootstrap.bundle.js et bootstrap.js), mais ont été épurés des commentaires et compressés pour les alléger et accélérer ainsi leur chargement.

Le dossier js/dist dans les sources contient les composants JavaScript dans des fichiers séparés. Ils peuvent être pratiques si vous utilisez peu de composants en évitant ainsi de tout charger avec la bibliothèque globale.

Il faut distinguer deux situations :

- Pendant la phase de développement, il est intéressant de pouvoir naviguer dans le code de Bootstrap, il faut donc mettre les fichiers non compressés.
- Lorsque votre site est en ligne, vous n'avez plus à vous inquiéter du code et seule compte la vitesse de chargement, il faut donc cette fois utiliser les versions compressées (min).

La figure suivante montre les fichiers utiles pendant la phase de développement (les fichiers bootstrap.js ou bootstrap.bundle.js et jquery.js ne sont à prévoir que si vous utilisez des plugins jQuery).



La figure suivante montre les fichiers utiles lorsque le site est en ligne (les fichiers bootstrap.min.js ou bootstrap.bundle.min.js et jquery.js ne sont à prévoir que si vous utilisez des plugins jQuery).



2. Intégration de bootstrap

a. Fichiers locaux

Pour que Bootstrap fonctionne, il faut que les pages HTML soient au format HTML 5, il faut donc prévoir le bon DOCTYPE :

```
<!DOCTYPE html>
```

```
<html lang="fr">
  ...
</html>
```

Il faut ensuite déclarer au minimum le fichier bootstrap.min.css (ou bootstrap.css) dans l'entête de la page web :

```
<head>
        …
    <link href="/css/bootstrap.min.css" rel="stylesheet">
</head>
```

À partir de là, toutes les classes sont accessibles… Évidemment vous devez adapter le lien selon la localisation de vos fichiers.
Si vous utilisez des composants JavaScript, vous devez également référencer la bibliothèque de Bootstrap ainsi que jQuery (la bibliothèque jQuery ne fait pas partie des fichiers téléchargés avec Bootstrap et doit être récupérée indépendamment sur http://jquery.com/). Il faut également prévoir d'ajouter la bibliothèque popper qui est nécessaire pour certains plugins :

```
<script src="/js/jquery.min.js"></script>
<script src="/js/popper.min.js"></script>
<script src="/js/bootstrap.min.js"></script>
```

Mais on peut aussi utiliser la version bundle qui inclut popper.js :

```
<script src="/js/jquery.min.js"></script>
<script src="/js/bootstrap.bundle.min.js"></script>
```

b. Les CDN

CDN est l'acronyme de « Content delivery network » ; c'est un réseau de serveurs qui met à disposition des bibliothèques. Il devient ainsi inutile de les prévoir sur son propre serveur, il suffit de « pointer » vers eux. Il y a des avantages à utiliser un CDN :

- libération de ressources et de la bande passante sur son propre serveur ;
- parallélisation des téléchargements (un CDN est sur plusieurs serveurs) ;
- accélération du chargement ;
- diminution de la latence ;
- actualisation automatique des bibliothèques ;
- amélioration du référencement…

Y a-t-il des inconvénients ?

D'après certains, l'utilisation d'un CDN, qui impose une requête DNS supplémentaire, ne serait judicieuse que si l'on a beaucoup de bibliothèques à charger. D'autre part si vous modifiez le fichier CSS de Bootstrap pour l'adapter à vos besoins, ce qui est très souvent le cas, vous ne pourrez plus bénéficier des avantages d'un CDN. Mais si vous voulez profiter de cette possibilité pour Bootstrap, utilisez de préférence bootstrap cdn. Vous pouvez évidemment utiliser un CDN également pour jQuery.

Si vous utilisez des CDN, l'installation en est d'autant simplifiée. Vous n'avez qu'à adapter les appels des bibliothèques :

```
1. <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/
bootstrap.min.css" integrity="sha384-Gn5384xqQ1aoWXA+058RXPxPg6fy4IWvTNh0E263XmFcJlSAwiGgFAW/
dAiS6JXm" crossorigin="anonymous">
2. …
```

```
3. <script src="https://code.jquery.com/jquery-3.2.1.slim.min.js" integrity="sha384-
KJ3o2DKtIkvYIK3UENzmM7KCkRr/rE9/Qpg6aAZGJwFDMVNA/GpGFF93hXpG5KkN" crossorigin="anonymous"></
script>
4. <script src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.12.9/
umd/popper.min.js" integrity="sha384-ApNbgh9B+Y1QKtv3Rn7W3mgPxhU9K/
ScQsAP7hUibX39j7fakFPskvXusvfa0b4Q" crossorigin="anonymous"></script>
5. <script src="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/
js/bootstrap.min.js" integrity="sha384-JZR6Spejh4U02d8jOt6vLEHfe/
JQGiRRSQQxSfFWpi1MquVdAyjUar5+76PVCmYl" crossorigin="anonymous"></script>
```

3. Une trame pour démarrer

a. La prestation minimale

Vous trouvez sur le site de Bootstrap ce template de base :

```
1. <!doctype html>
2. <html lang="fr">
3.   <head>
4.     <!-- Required meta tags -->
5.     <meta charset="utf-8">
6.     <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
7.     <!-- Bootstrap CSS -->
8.     <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/
bootstrap.min.css" integrity="sha384-Gn5384xqQ1aoWXA+058RXPxPg6fy4IWvTNh0E263XmFcJlSAwiGgFAW/
dAiS6JXm" crossorigin="anonymous">
9.     <title>Hello, world!</title>
10.   </head>
11.   <body>
12.     <h1>Hello, world!</h1>
13.     <!-- Optional JavaScript -->
14.     <!-- jQuery first, then Popper.js, then Bootstrap JS -->
15.     <script src="https://code.jquery.com/jquery-3.2.1.slim.min.js" integrity="sha384-
KJ3o2DKtIkvYIK3UENzmM7KCkRr/rE9/Qpg6aAZGJwFDMVNA/GpGFF93hXpG5KkN" crossorigin="anonymous"></
script>
16.     <script src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.12.9/
umd/popper.min.js" integrity="sha384-ApNbgh9B+Y1QKtv3Rn7W3mgPxhU9K/
ScQsAP7hUibX39j7fakFPskvXusvfa0b4Q" crossorigin="anonymous"></script>
17.     <script src="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/
js/bootstrap.min.js" integrity="sha384-JZR6Spejh4U02d8jOt6vLEHfe/
JQGiRRSQQxSfFWpi1MquVdAyjUar5+76PVCmYl" crossorigin="anonymous"></script>
18.   </body>
19. </html>
```

Pourquoi y a-t-il un attribut d'intégrité pour le chargement des fichiers par les CDN ?

Les CDN sont pratiques, mais il faut être sûr de charger effectivement ce que l'on a demandé. Si un pirate modifie les

fichiers du serveur, on pourrait avoir des surprises. Le contrôle d'intégrité permet d'être sûr d'obtenir le bon fichier.

Dans le template on trouve cette ligne :

```
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
```

b. Un exemple

Vous pouvez trouver sur le site de nombreux templates d'exemple. Les éléments nécessaires à la compréhension de ces exemples seront exposés tout au long de ce cours. Voyons toutefois le premier, nommé Starter Template, qui est le plus classique :

```
1. <!doctype html>
2. <html lang="en">
3.   <head>
4.     <meta charset="utf-8">
5.     <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
6.     <meta name="description" content="">
7.     <meta name="author" content="">
8.     <link rel="icon" href="../../../../favicon.ico">
9.
10.     <title>Starter Template for Bootstrap</title>
11.
12.     <!-- Bootstrap core CSS -->
13.     <link href="../../../../dist/css/bootstrap.min.css" rel="stylesheet">
14.
15.     <!-- Custom styles for this template -->
16.     <link href="starter-template.css" rel="stylesheet">
17.   </head>
18.
19.   <body>
20.
21.     <nav class="navbar navbar-expand-md navbar-dark bg-dark fixed-top">
22.       <a class="navbar-brand" href="#">Navbar</a>
23.       <button class="navbar-toggler" type="button" data-toggle="collapse" data-
target="#navbarsExampleDefault" aria-controls="navbarsExampleDefault" aria-expanded="false" aria-
label="Toggle navigation">
24.         <span class="navbar-toggler-icon"></span>
25.       </button>
26.
27.       <div class="collapse navbar-collapse" id="navbarsExampleDefault">
28.         <ul class="navbar-nav mr-auto">
29.           <li class="nav-item active">
30.             <a class="nav-link" href="#">Home <span class="sr-only">(current)</span></a>
31.           </li>
32.           <li class="nav-item">
33.             <a class="nav-link" href="#">Link</a>
34.           </li>
```

```
35.           <li class="nav-item">
36.             <a class="nav-link disabled" href="#">Disabled</a>
37.           </li>
38.           <li class="nav-item dropdown">
39.             <a class="nav-link dropdown-toggle" href="http://
example.com" id="dropdown01" data-toggle="dropdown" aria-haspopup="true" aria-
expanded="false">Dropdown</a>
40.             <div class="dropdown-menu" aria-labelledby="dropdown01">
41.               <a class="dropdown-item" href="#">Action</a>
42.               <a class="dropdown-item" href="#">Another action</a>
43.               <a class="dropdown-item" href="#">Something else here</a>
44.             </div>
45.           </li>
46.         </ul>
47.         <form class="form-inline my-2 my-lg-0">
48.           <input class="form-control mr-sm-2" type="text" placeholder="Search" aria-
label="Search">
49.           <button class="btn btn-outline-success my-2 my-sm-0" type="submit">Search</button>
50.         </form>
51.       </div>
```

```
52.     </nav>
53.     <main role="main" class="container">
54.       <div class="starter-template">
55.         <h1>Bootstrap starter template</h1>
56.         <p class="lead">Use this document as a way to quickly start any new project.<br> All
 you get is this text and a mostly barebones HTML document.</p>
57.       </div>
58.     </main><!-- /.container -->
59.     <!-- Bootstrap core JavaScript
60.     ================================================== -->
61.     <!-- Placed at the end of the document so the pages load faster -->
62.     <script src="https://code.jquery.com/jquery-3.2.1.slim.min.js" integrity="sha384-
KJ3o2DKtIkvYIK3UENzmM7KCkRr/rE9/Qpg6aAZGJwFDMVNA/GpGFF93hXpG5KkN" crossorigin="anonymous"></
script>
63.     <script>window.jQuery || document.write('<script src="../../../../assets/js/vendor/
jquery-slim.min.js"><\/script>')</script>
64.     <script src="../../../../assets/js/vendor/popper.min.js"></script>
65.     <script src="../../../../dist/js/bootstrap.min.js"></script>
66.   </body>
67. </html>
```

On a le résultat suivant :



On retrouve ce qu'on a vu précédemment et l'appel au fichier CSS de Bootstrap. On trouve la référence d'un autre fichier CSS : starter-template.css. Voyons ce que contient ce fichier :

```
1. body {
2.   padding-top: 5rem;
3. }
4. .starter-template {
5.   padding: 3rem 1.5rem;
6.   text-align: center;
7. }
```

Nous verrons, lorsque nous parlerons du composant « Barre de navigation » de Bootstrap, qu'il faut parfois réserver un espace en haut de la page (avec padding-top : 5rem) pour éviter que le texte du début ne se retrouve sous la barre. On trouve aussi ici la classe starter-template qui se contente de fixer un espace interne et un alignement centré du texte. Cette classe ne sert que pour le texte de présentation du template et peut être supprimée ou modifiée selon votre convenance.

On trouve aussi dans la page une barre de navigation.

```
1. <nav class="navbar navbar-expand-md navbar-dark bg-dark fixed-top">
2.   <a class="navbar-brand" href="#">Navbar</a>
3.   <button class="navbar-toggler" type="button" data-toggle="collapse" data-
target="#navbarsExampleDefault" aria-controls="navbarsExampleDefault" aria-expanded="false" aria-
label="Toggle navigation">
4.     <span class="navbar-toggler-icon"></span>
5.   </button>
6.   <div class="collapse navbar-collapse" id="navbarsExampleDefault">
7.     <ul class="navbar-nav mr-auto">
8.       <li class="nav-item active">
```

```
9.          <a class="nav-link" href="#">Home <span class="sr-only">(current)</span></a>
10.       </li>
11.       <li class="nav-item">
12.         <a class="nav-link" href="#">Link</a>
13.       </li>
14.       <li class="nav-item">
15.         <a class="nav-link disabled" href="#">Disabled</a>
16.       </li>
17.       <li class="nav-item dropdown">
18.         <a class="nav-link dropdown-toggle" href="http://example.com" id="dropdown01" data-
toggle="dropdown" aria-haspopup="true" aria-expanded="false">Dropdown</a>
19.         <div class="dropdown-menu" aria-labelledby="dropdown01">
20.           <a class="dropdown-item" href="#">Action</a>
21.           <a class="dropdown-item" href="#">Another action</a>
22.           <a class="dropdown-item" href="#">Something else here</a>
23.         </div>
24.       </li>
25.     </ul>
26.     <form class="form-inline my-2 my-lg-0">
27.       <input class="form-control mr-sm-2" type="text" placeholder="Search" aria-
label="Search">
28.       <button class="btn btn-outline-success my-2 my-sm-0" type="submit">Search</button>
29.     </form>
30.   </div>
31. </nav>
```

Le contenu de la page est ensuite inséré dans une balise <main> comportant la classe container :

```
1. <main role="main" class="container">
2.   <div class="starter-template">
3.     <h1>Bootstrap starter template</h1>
4.     <p class="lead">Use this document as a way to quickly start any new project.<br> All you
 get is this text and a mostly barebones HTML document.</p>
5.   </div>
6. </main><!-- /.container -->
```

4. Flexbox

Les flexbox sont devenus de plus en plus populaires depuis quelques années et leur utilisation grandit de plus en plus. Mais de quoi s'agit-il ? Avec le CSS on a un modèle de boîtes (de type block ou inline) qui permet la mise en page. Vous avez sans doute l'habitude du positionnement flottant qui permet de jouer avec la position des blocs. Avec les flexbox les boîtes deviennent flexibles. On a un conteneur et des éléments, et le conteneur a la capacité

de modifier les dimensions, positions et l'ordre des éléments pour qu'ils s'adaptent aux contraintes d'affichage. En fait exactement ce qui manque cruellement avec le modèle de boîtes classique, on peut même dire rigide par rapport à ce modèle flexible.

Dans sa version 3 Bootstrap n'utilisait pas flexbox, mais cela change avec la version 4 parce que désormais il n'utilise que ça !

Mais je précise que Bootstrap est équipé de nombreuses classes qui facilitent l'utilisation de flexbox et je vais les citer dans ce chapitre au fur et à mesure de la présentation des superbes possibilités qui nous sont offertes.

Faut-il vraiment connaître flexbox pour utiliser Bootstrap ?

Ce n'est pas indispensable pour une utilisation de base, mais ça le devient très vite dès qu'on veut utiliser plus efficacement la bibliothèque.

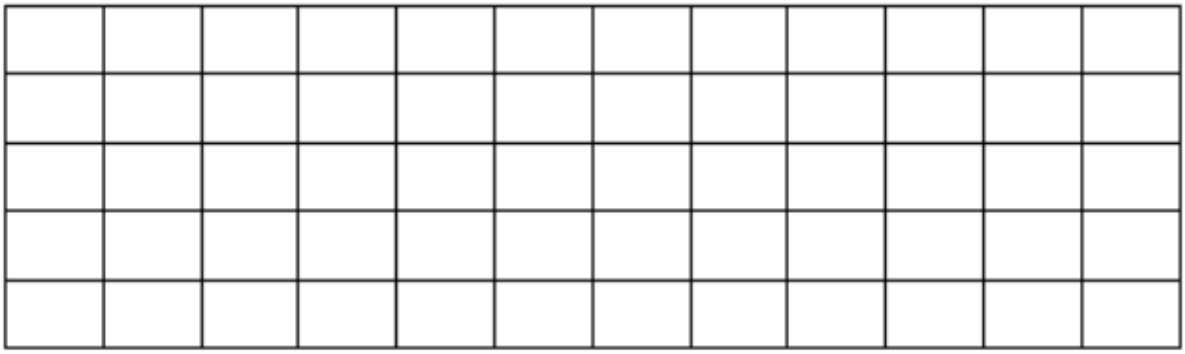
III. Présentation de la grille

L'organisation spatiale des pages web est l'une des premières préoccupations lorsque l'on crée un site web. Prévoit-on une bannière ? Faut-il un espace pour un menu à gauche ou en haut ? Y aura-t-il des blocs sur un des côtés pour recevoir certaines fonctionnalités comme la connexion ou des infos ? Faut-il prévoir un bas de page ?

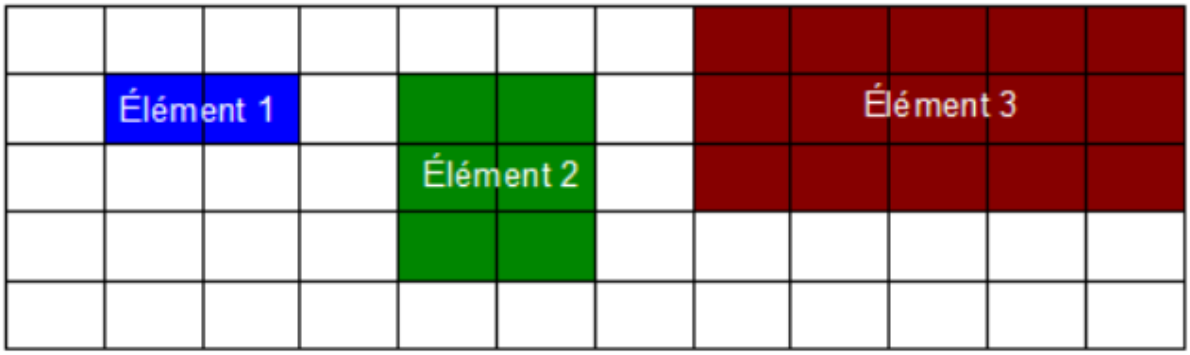
Bootstrap ne répond évidemment pas à ces questions, mais il peut grandement vous faciliter la tâche, avec son système de grille, pour obtenir le résultat que vous souhaitez.

1. Le principe d'une grille

Une grille est tout simplement un découpage en cellules :



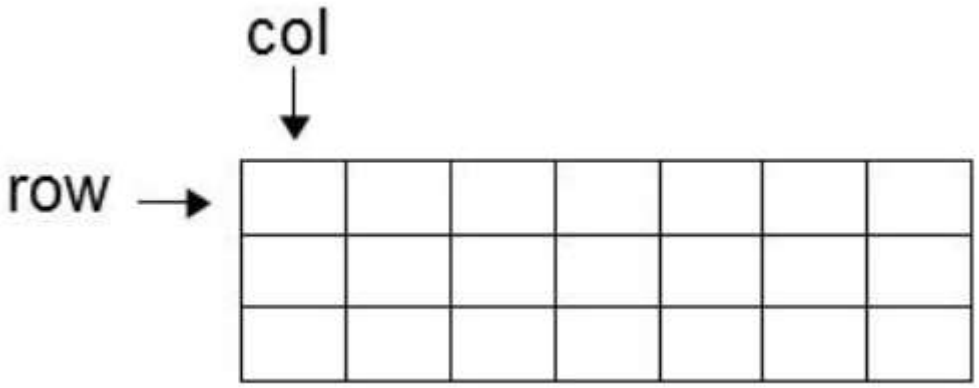
On peut alors décider d'organiser du contenu en utilisant pour chaque élément une ou plusieurs cellules :



La grille de Bootstrap comporte douze colonnes comme dans l'illustration précédente. Vous commencez sans doute à comprendre l'utilité de cette organisation...

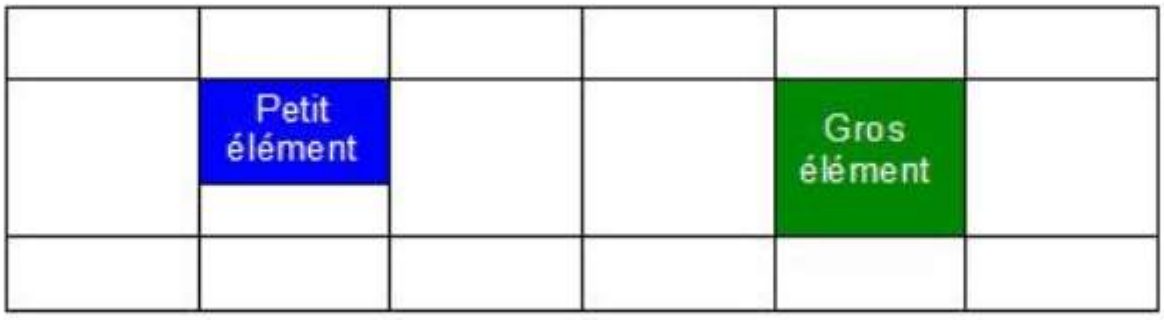
2. Terminologie

Une grille est découpée en rangées (appelées ***row***, parce que tout est en anglais) et colonnes (col).



3. La grille de Bootstrap

La grille de Bootstrap n'est pas aussi idéale que celle présentée précédemment. Le découpage en colonnes est tout simplement une division en pourcentage de la largeur de la fenêtre de visualisation et correspond donc à ce qu'on vient de voir. En revanche, il n'en est pas vraiment de même pour les rangées. Ces dernières ont la hauteur de leur contenu (voir figure suivante).



Moralité : une rangée prend la hauteur du plus gros élément qu'elle contient.

Puisque la largeur des colonnes est contrainte, le flux des données s'écoule verticalement, ce qui est un comportement HTML classique. Il faudra en tenir compte lors de la mise en page.

4. Organisation de la grille

Bootstrap est essentiellement un fichier CSS. Il comporte de nombreuses classes que l'on peut utiliser directement
dans les balises HTML.

La première classe à connaître est **row**, qui représente une rangée. Cette classe établit des marges négatives à droite et à gauche pour équilibrer les marges de séparation des colonnes qui ont la même valeur, mais évidemment en positif :

```
1. .row {
2.   ...
3.   margin-right: -15px;
4.   margin-left: -15px;
5. }
```

Il faut ensuite définir le nombre de colonnes pour chaque élément en sachant qu'il y en a au maximum douze. Pour définir le nombre de colonnes utilisées pour chaque élément, on dispose de cinq batteries de douze classes :

- ***une colonne* : col-1 ou col-sm-1 ou col-md-1 ou col-lg-1 ou col-xl-1 ;**
- ***deux colonnes* : col-2 ou col-sm-2 ou col-md-2 ou col-lg-2 ou col-xl-2 ;**
- **...**
- ***douze colonnes* : col-12 ou col-sm-12 ou col-md-12 ou col-lg-12 ou col-xl-12.**

Pour disposer d'une marge entre les colonnes, ces classes prévoient un padding de 15px.

Pourquoi cinq sortes de classes pour les colonnes ?

Je vous ai déjà dit que Bootstrap est « responsive », ce qui veut dire qu'il s'adapte à la taille de l'écran. Il permet une visualisation aussi bien sur un écran géant que sur un smartphone.

Est-ce qu'il est possible de supprimer toutes ces marges ?

Oui. Il suffit d'utiliser la classe no-gutters avec la classe row. Ce qui a pour effet de supprimer toutes les marges à la fois ainsi que les lignes et les colonnes.

Mais que se passe-t-il pour les éléments d'une page web lorsque la fenêtre diminue ou s'élargit ?

On peut envisager deux hypothèses : les éléments se redimensionnent en restant positionnés, ou alors ils s'empilent quand la fenêtre devient plus étroite et se positionnent côte à côte quand elle s'élargit. Voici à la figure suivante l'exemple d'une partie de la page d'accueil du site colly.com qui illustre ce phénomène avec trois situations (grand écran, écran moyen et petit écran) :



Bottled for your pleasure

POTTED AUTOBIOGRAPHY



Hello. I'm a designer. I also do plenty of writing, speaking, and sleeping. I'm based in Nottingham, England, but my work means I'm often found in all sorts of other fine places. Please read on →

Dropping science like it's hot

THE SPLENDID JOURNAL

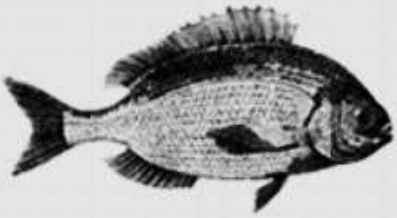


§ 2014 music roundup

It was yet another cracking year for new music, and I consumed as much as possible, making my annual roundup as difficult as ever. To follow my 2010, 2011,… More →

Catalogued nocturnal matter

EXHAUSTIVE ARCHIVES



866—2013: a year in review

865—2013 in music

868—Waking in Florence

863—Yet another year in review

862—Yet another year in music

Mr. Collison is currently

RECEIVING YOUR EMAILS

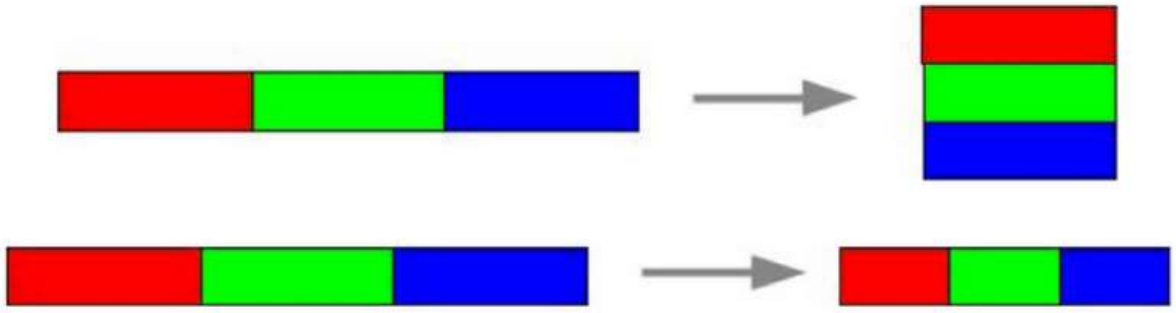


Opinions & queries this way

Drop me a line if you wish. I'm always interested in new projects and opportunities and will of course do my best to reply to your emails swiftly and efficiently. →



Pour le moment on va juste se demander comment on peut choisir entre les deux options présentées à la figure suivante.



C'est ici qu'interviennent les cinq sortes de classes vues précédemment pour les colonnes. Bootstrap considère cinq sortes de médias :

- les très petits genre smartphones en mode portrait (moins de 576px) ;
- les petits genre smartphones en mode paysage (plus de 576px, mais moins de 768px) ;
- les moyens genre tablettes (plus de 768px, mais moins de 992px) ;
- les grands (plus de 992px, mais moins de 1200px) ;
- les très grands (plus de 1200px).

Vous trouverez à la figure suivante le tableau de la documentation officielle pour illustrer les différences de réaction selon la catégorie et les classes associées.

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	Extra large ≥1200px
Max container width	None (auto)	540px	720px	960px	1140px
Class prefix	.col-	.col-sm-	.col-md-	.col-lg-	.col-xl-
# of columns	12				
Gutter width	30px (15px on each side of a column)				
Nestable	Yes				
Column ordering	Yes				

Le nom des classes est intuitif :

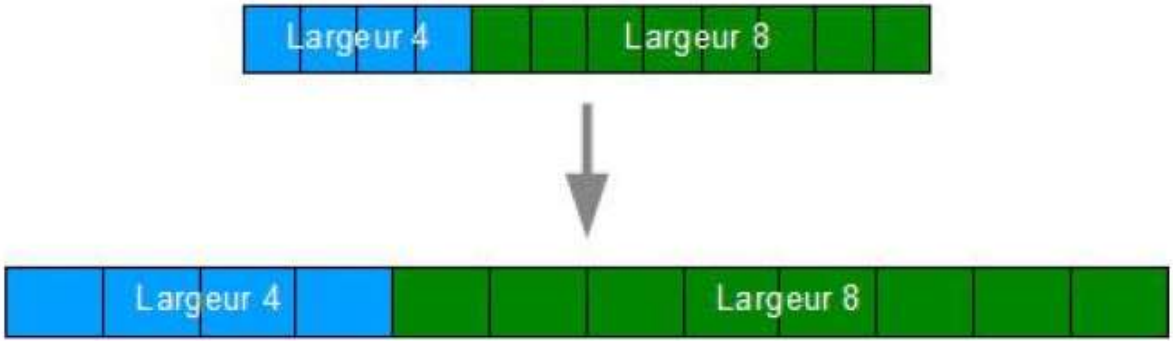
- rien pour les très petits écrans (valeur de référence) ;
- sm pour small ;
- md pour medium ;
- lg pour large ;
- xl pour x-large.

Exemple

Pour avoir un élément de quatre colonnes de large accouplé avec un élément de huit colonnes de large sur un smartphone en mode portrait (et donc aussi tous les formats au-dessus) on a :

```
1. <div class="row">
2.     <div class="col-4">Largeur 4</div>
3.     <div class="col-8">Largeur 8</div>
4. </div>
```

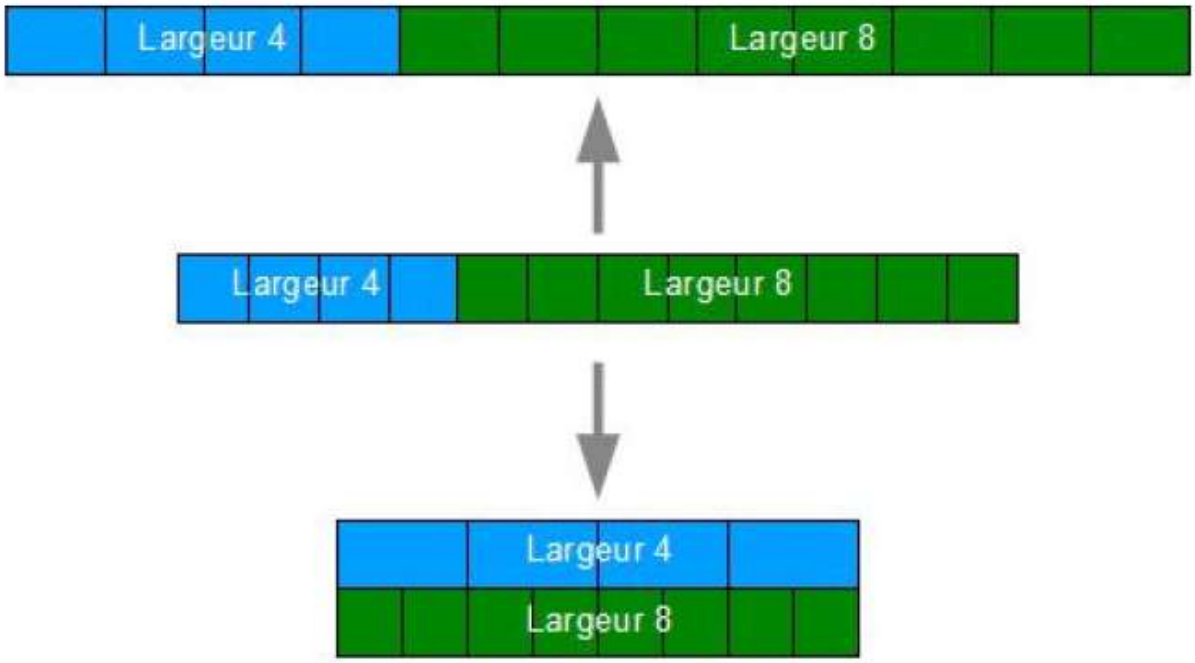
La figure suivante illustre ce code avec l'effet quand on passe sur un écran plus grand.



Voici la version « tablette » :

```
1. <div class="row">
2.     <div class="col-md-4">Largeur 4</div>
3.     <div class="col-md-8">Largeur 8</div>
4. </div>
```

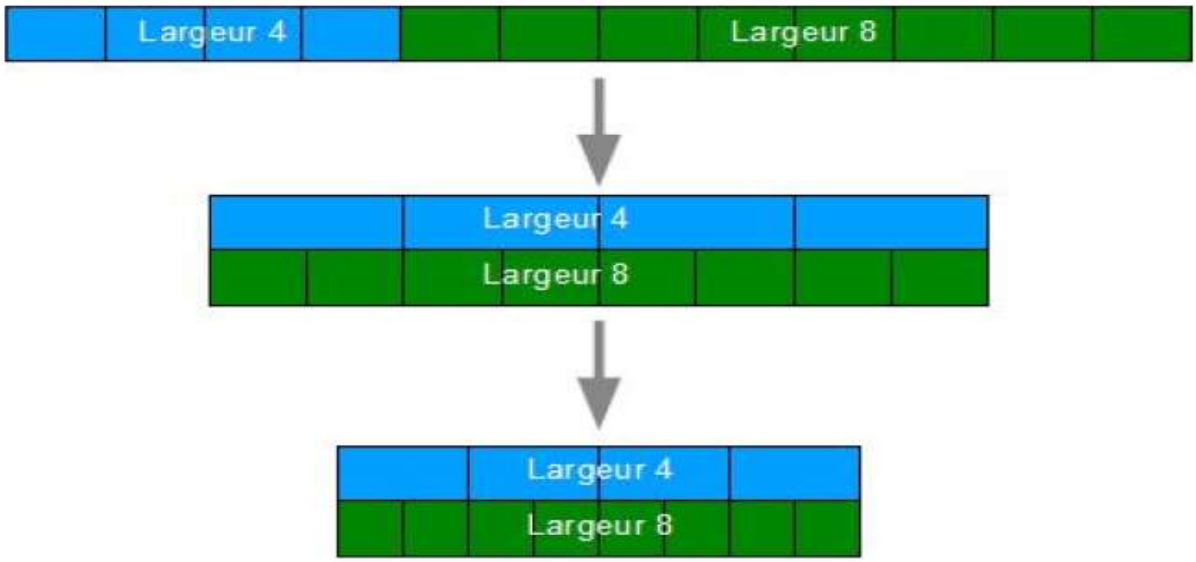
La figure suivante illustre ce code avec l'effet quand on passe sur un écran plus petit ou plus grand.



Vous remarquez que lors de l'empilage, les colonnes prennent toute la place disponible.

Et enfin, voici la version très grand écran :

```
1. <div class="row">
2.     <div class="col-lg-4">Largeur 4</div>
3.     <div class="col-lg-8">Largeur 8</div>
4. </div>
```



5. Le conteneur

Une grille doit toujours se placer dans un conteneur.

Bootstrap propose deux types de conteneurs selon l'effet désiré :

- un conteneur de largeur fixe (qui s'adapte évidemment selon la largeur de l'écran utilisé) :



La classe à utiliser est container :

```
<div class="container">
  <!-- le contenu ici -->
</div>
```

- un conteneur à largeur « fluide » qui occupe toujours toute la largeur disponible :



La classe à utiliser est container-fluid :

```
<div class="container-fluid">
  <!-- Le contenu ici -->
</div>
```

Le centrage du conteneur est fait de façon classique avec de petites marges internes de 15px et les marges droite et gauche automatiques :

```
1. .container {
2.   width: 100%;
3.   padding-right: 15px;
4.   padding-left: 15px;
5.   margin-right: auto;
6.   margin-left: auto;
7. }
```

Ces règles sont communes aux deux types de conteneurs. La différence consiste à ajouter des règles pour le conteneur fixe pour tenir compte du média de visualisation utilisé :

```
// Smartphones en mode paysage
@media (min-width: 576px) {
  .container {
    max-width: 540px;
  }
}
// Tablettes
@media (min-width: 768px) {
  .container {
    max-width: 720px;
  }
}
// Grand écran
@media (min-width: 992px) {
  .container {
    max-width: 960px;
  }
}
// Très grand écran
@media (min-width: 1200px) {
  .container {
    max-width: 1140px;
  }
}
```

Exemple

Après cette petite mise en jambe pour vous présenter la notion de grille dans Bootstrap, on va passer en revue les possibilités dans le détail dans les chapitres suivants. Mais pour visualiser tous les exemples qui vont suivre, je vous propose d'ajouter un peu de style, histoire d'afficher les éléments de façon explicite. Pour éviter de polluer le code HTML on va mettre ce style spécifique dans un fichier CSS particulier (ce fichier sera nommé tuto.css dans le code) :

```
body {
  padding-top: 10px;
}
[class*="col"], footer, aside {
  background-color: lightblue;
  border: 1px solid blue;
  border-radius: 4px;
  line-height: 40px;
  text-align: center;
}
```

Le seul but de ces règles de style est de faire apparaître nettement les éléments à l'écran.

Une petite marge interne en haut pour le corps, pour éviter que tout soit collé en haut de l'écran. Des bordures (avec des coins arrondis pour l'esthétique) et un fond coloré pour distinguer les éléments. Une hauteur fixée à 40 pixels, parce qu'étant donné que la hauteur des rangées dépend du contenu, on aurait un rendu hétéroclite.

Considérons un petit exemple pour se rendre compte du rendu :

```
1. <!DOCTYPE html>
2. <html lang="fr">
3.   <head>
4.     <meta charset="utf-8">
5.     <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
6.     <link rel="stylesheet" href="css/bootstrap.css">
7.     <link rel="stylesheet" href="css/tuto.css">
8.   </head>
9.   <body>
10.     <div class="container">
11.       <div class="row">
12.         <div class="col-sm-2">
13.           2 colonnes
14.         </div>
15.         <div class="col-sm-3">
16.           3 colonnes
17.         </div>
18.         <div class="col-sm-7">
19.           7 colonnes
20.         </div>
21.       </div>
22.     </div>
23.   </body>
24. </html>
```



6. La grille en action

Exemple 1 :

Une seule rangée avec des largeurs égales et variables.

```
1. <!DOCTYPE html>
2. <html lang="fr">
3.   <head>
4.     <meta charset="utf-8">
5.     <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
6.     <link rel="stylesheet" href="css/bootstrap.css">
7.     <link rel="stylesheet" href="css/tuto.css">
8.   </head>
9.   <body>
10.     <div class="container">
11.       <div class="row">
12.         <div class="col">
13.           6 col
14.         </div>
15.         <div class="col">
16.           6 col
17.         </div>
18.       </div>
19.     </div>
20.   </body>
21. </html>
```

Voici le résultat :



Si on ne veut pas des largeurs égales on peut spécifier le nombre de colonnes :

```
1. <div class="row">
2.   <div class="col">
3.     2 col
4.   </div>
5.   <div class="col-8">
6.     8 col
7.   </div>
8.   <div class="col">
9.     2 col
10.   </div>
11. </div>
```

Exemple 2 : Plusieurs rangées

Pour obtenir plusieurs rangées, il suffit d'utiliser plusieurs fois la classe row. Un petit exemple avec trois rangées pour voir l'effet :

```
1. <div class="row">
2.   <div class="col-1">1 col</div>
3.   <div class="col-2">2 colonnes</div>
4.   <div class="col-3">3 colonnes</div>
5.   <div class="col">6 colonnes</div>
6. </div>
7. <div class="row">
8.   <div class="col">12 colonnes</div>
9. </div>
10. <div class="row">
11.   <div class="col-4">4 colonnes</div>
12.   <div class="col">8 colonnes</div>
13. </div>
```

Voici le résultat :



Exemple 3 : largeur variable

Par défaut les colonnes ont toujours la même largeur, mais le passage à flexbox permet de jouer facilement sur des différences de largeur. Mais pour le coup on peut perdre un peu la notion de colonne. Regardez cet exemple :

```
1. <div class="row">
2.   <div class="col">
3.     Répartition du reste
4.   </div>
5.   <div class="col col-auto">
6.     Contenu auto
7.   </div>
8.   <div class="col">
9.     Répartition du reste
10.   </div>
11. </div>
```

Voici le résultat :



L'utilisation de la classe col-auto fixe les règles de flexbox pour que l'élément s'adapte au contenu (auto) :

Exemple 4 : Éléments égaux sur plusieurs rangées

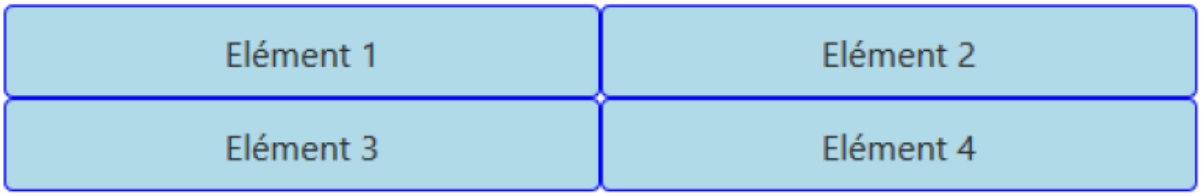
```
1. <div class="row">
2.   <div class="col">
3.     Elément 1
4.   </div>
5.   <div class="col">
6.     Elément 2
7.   </div>
8.   <div class="col">
9.     Elément 3
10.  </div>
11.  <div class="col">
12.    Elément 4
13.  </div>
14. </div>
```

Voici le résultat :



Si je les veux sur deux rangées, je peux définir un nombre de colonnes parce que si on dépasse les douze colonnes on passe automatiquement sur une nouvelle rangée (wrapping) :

```
1. <div class="row">
2.   <div class="col-6">
3.     Elément 1
4.   </div>
5.   <div class="col-6">
6.     Elément 2
7.   </div>
8.   <div class="col-6">
9.     Elément 3
10.  </div>
11.  <div class="col-6">
12.    Elément 4
13.  </div>
14. </div>
```



Mais il existe aussi la classe w-100 pour obtenir le même effet :

```
1. <div class="row">
2.   <div class="col">
3.     Elément 1
4.   </div>
5.   <div class="col">
```

```
6.      Elément 2
7.    </div>
8.    <div class="w-100"></div>
9.    <div class="col">
10.      Elément 3
11.    </div>
12.    <div class="col">
13.      Elément 4
14.    </div>
15. </div>
```

w-100 signifie tout simplement pour **width 100%**. Il existe aussi w-25, w-50 et w-75.

Exemple 5 - Saut de colonnes

Bootstrap 3 permettait de sauter des colonnes avec des classes ***d'offset.*** Ces classes ont disparu avec la version 4. Désormais on utilise des utilitaires de marges qui apportent moins de souplesse, mais plus de simplicité.

La classe est composée de trois éléments séparés par des traits d'union : *{propriété}{côtés}-{support optionnel}-auto*

1. La propriété définit le type :

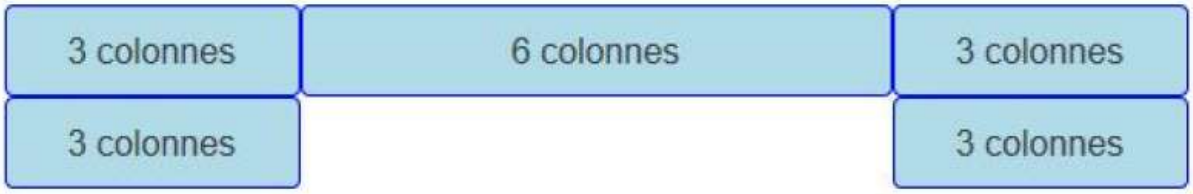
- m : marge
- p : remplissage (padding)

2. Les côtés définissent la position :

- t : haut (top)
- r : droite (right)
- b : bas (bottom)
- l : gauche (left)
- x : gauche et droite (axe x)
- y : haut et bas (axe y)

Par exemple la classe ml-auto définit une marge automatique à gauche alors que la classe mr-auto fait la même chose pour la droite.

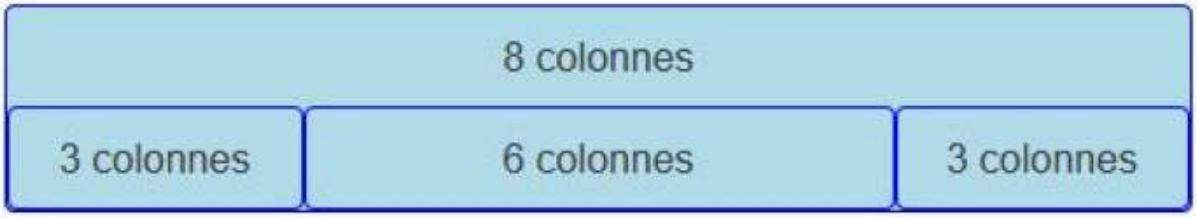
```
1. <div class="row">
2.   <div class="col-3">3 colonnes</div>
3.   <div class="col">6 colonnes</div>
4.   <div class="col-3">3 colonnes</div>
5. </div>
6. <div class="row">
7.   <div class="col-3">3 colonnes</div>
8.   <div class="col-3 ml-auto">3 colonnes</div>
9. </div>
```



- Exemple 6 - Imbrication d'éléments

On doit souvent imbriquer des éléments dans une page web, est-ce possible avec Bootstrap ? Autrement dit, inclure un row dans un col.

```
1. <div class="row">
2.   <div class="col-8">8 colonnes
3.     <div class="row">
4.       <div class="col-3">3 colonnes</div>
5.       <div class="col">6 colonnes</div>
6.       <div class="col-3">3 colonnes</div>
7.     </div>
8.   </div>
9. </div>
```



Les imbrications peuvent s'enchaîner pour répondre à des besoins complexes :

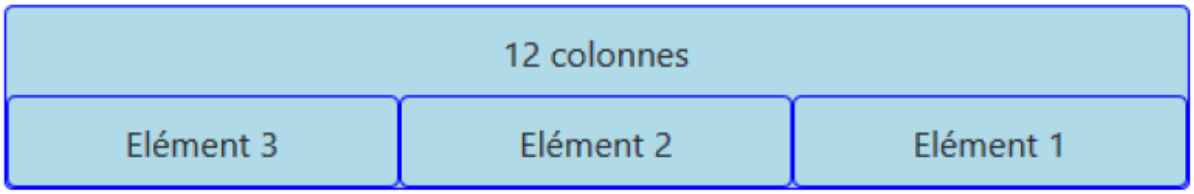
```
1. <div class="row">
2.   <div class="col-12">Premier niveau avec 12 colonnes
3.     <div class="row">
4.       <div class="col-8">Second niveau avec 8 colonnes
5.         <div class="row">
6.           <div class="col-4">Troisième niveau avec 4 colonnes</div>
7.             <div class="col-6 ml-auto">Troisième niveau avec 6 colonnes
8.               <div class="row">
9.                 <div class="col-3 mx-auto">Quatrième niveau avec 3 colonnes</div>
10.                 <div class="col-5 mx-auto">Quatrième niveau avec 5 colonnes</div>
11.               </div>
12.             </div>
13.           </div>
14.         </div>
15.       <div class="col">Second niveau avec 4 colonnes</div>
16.     </div>
17.   </div>
18. </div>
```



Exemple 7 - Ordre des colonnes

Très logiquement, les colonnes s'affichent dans l'ordre du flux. Il peut arriver qu'on veuille bouleverser cet ordre.

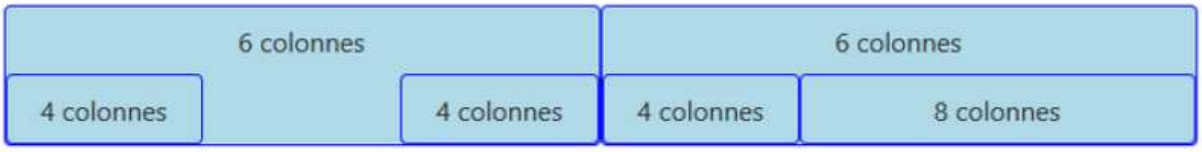
```
1. <div class="row">
2.   <div class="col">12 colonnes
3.     <div class="row">
4.       <div class="col order-3">Elément 1</div>
5.       <div class="col order-2">Elément 2</div>
6.       <div class="col order-1">Elément 3</div>
7.     </div>
8.   </div>
9. </div>
```



Les classes ***order-**** permettent de fournir un ordre d'affichage avec une valeur allant de 1 à 12.

Il existe aussi les classes order-sm-, order-md-*, order-lg-* et order-xl-*.*

Exercice : Ecrire le code permettant d'obtenir le résultat suivant.



IV. Mise en page

L'intérêt principal d'une grille est de réaliser une mise en page. Maintenant que vous savez comment utiliser la grille de Bootstrap, nous allons voir dans ce chapitre quelques exemples pour structurer correctement une page.

1. Mise en page simple

Commençons par quelque chose de simple avec un entête, un menu à gauche, une section à droite et un pied de page :

```
<div class="container">
  <header class="row">
    <div class="col">
      Entete
    </div>
  </header>
  <div class="row">
    <nav class="col-lg-2">
      Menu
    </nav>
    <section class="col-lg-10">
      Section
    </section>
  </div>
  <footer class="row">
    <div class="col">
      Pied de page
    </div>
  </footer>
</div>
```

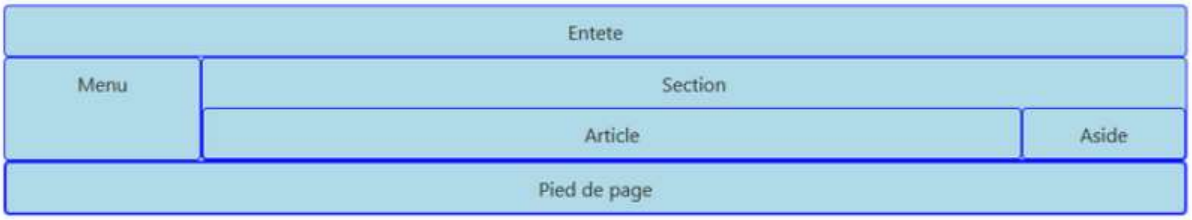
Le résultat se trouve à la figure suivante.



2. Mise en page plus chargée

Considérons maintenant un cas un peu plus riche :

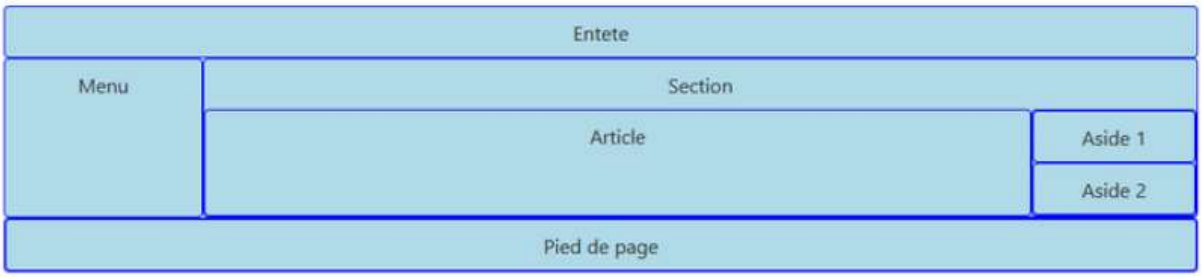
```
1. <div class="container">
2.   <header class="row">
3.     <div class="col">
4.       Entete
5.     </div>
6.   </header>
7.   <div class="row">
8.     <nav class="col-md-2">
9.       Menu
10.    </nav>
11.    <section class="col-md-10">
12.      Section
13.      <div class="row">
14.        <article class="col-lg-10">
15.          Article
16.        </article>
17.        <aside class="col-lg-2">
18.          Aside
19.        </aside>
20.      </div>
21.    </section>
22.  </div>
23.  <footer class="row">
24.    <div class="col">
25.      Pied de page
26.    </div>
27.  </footer>
28. </div>
```



3. Plusieurs asides

Enrichissons encore la mise en page :

```
<div class="container">
  <header class="row">
    <div class="col">
      Entete
    </div>
  </header>
  <div class="row">
    <nav class="col-md-2">
      Menu
    </nav>
    <section class="col">
      Section
      <div class="row">
        <article class="col-lg-10">
          Article
        </article>
        <div class="col">
          <div class="row">
            <aside class="col">
              Aside 1
            </aside>
            <div class="w-100"></div>
            <aside class="col">
              Aside 2
            </aside>
          </div>
        </div>
      </div>
    </section>
  </div>
  <footer class="row">
    <div class="col">
      Pied de page
    </div>
  </footer>
</div>
```



➢ Mise en forme des images

Bootstrap propose trois types de mise en forme pour les images :

- Image avec coins arrondis ;
- Image en cercle ;
- Image dans un cadre

EXAMPLE








```
<img src="..." alt="..." class="img-rounded">
<img src="..." alt="..." class="img-circle">
<img src="..." alt="..." class="img-thumbnail">
```

Copy

V. Mise en forme des tableaux

Bootstrap propose 5 types de présentation des tableaux :

1. Les tableaux simples en pleine largeur

Pour avoir un tableau simple en utilisant le Bootstrap, il vous faut utiliser la classe .table qui assure une mise en forme sobre du tableau. Cela implique que le tableau à comme élément parent <body>, sa largeur sera celle du <body>. Donc si besoin est, vous pouvez placer vos tableaux dans des boites à largeur fixe.

Exemple :

EXAMPLE

#	First Name	Last Name	Username
1	Mark	Otto	@mdo
2	Jacob	Thornton	@fat
3	Larry	the Bird	@twitter

```
<table class="table">
  ...
</table>
```

2. Les tableaux à lignes alternées

Pour avoir des tableaux avec des lignes grises et blanches, il suffit d'ajouter la classe **.table-striped.** Le fond grisé est appliqué à la balise <td> avec le sélecteur :nth-child(odd).

```
.table-striped >tbody>tr:nth-child(odd) {
background-color: #f9f9f9;
}
```

Exemple :

EXAMPLE

#	First Name	Last Name	Username
1	Mark	Otto	@mdo
2	Jacob	Thornton	@fat
3	Larry	the Bird	@twitter

```
<table class="table table-striped">
  ...
</table>
```

3. Les tableaux encadrés

Pour avoir des tableaux encadrés, ajoutez la classe **.table-bordered** à l'élément <table>.

Exemple :

EXAMPLE

#	First Name	Last Name	Username
1	Mark	Otto	@mdo
	Mark	Otto	@TwBootstrap
2	Jacob	Thornton	@fat
3	Larry the Bird		@twitter

```
<table class="table table-bordered">
  ...
</table>
```

4. Les tableaux avec effet de survol

Pour avoir un tableau plus resserré (valeur de la propriété padding réduite), ajoutez la classe **.table-condensed** à l'élément <table>.

```
.table-condensed >thead>tr>th,
.table-condensed >tbody>tr>th,
.table-condensed >tfoot>tr>th,
.table-condensed >thead>tr> td,
.table-condensed >tbody>tr> td,
.table-condensed >tfoot>tr> td {
padding: 5px;
}
```

Exemple :

EXAMPLE

#	First Name	Last Name	Username
1	Mark	Otto	@mdo
2	Jacob	Thornton	@fat
3	Larry the Bird		@twitter

```
<table class="table table-condensed">
  ...
</table>
```

5. Les tableaux avec effet de survol

Pour activer une mise en évidence des lignes survolées par le pointeur de la souris, ajoutez la classe **.table-hover**à l'élément <table>.

```
.table-hover >tbody>tr:hover {
background-color: #f5f5f5;
}
```

Remarque

Pour avoir un tableau adaptatif, optimisé pour un affichage sur mobile avec des largeurs d'écran inférieur à 768px, placer le tableau dans une boite et associer la classe **.table-responsive**.

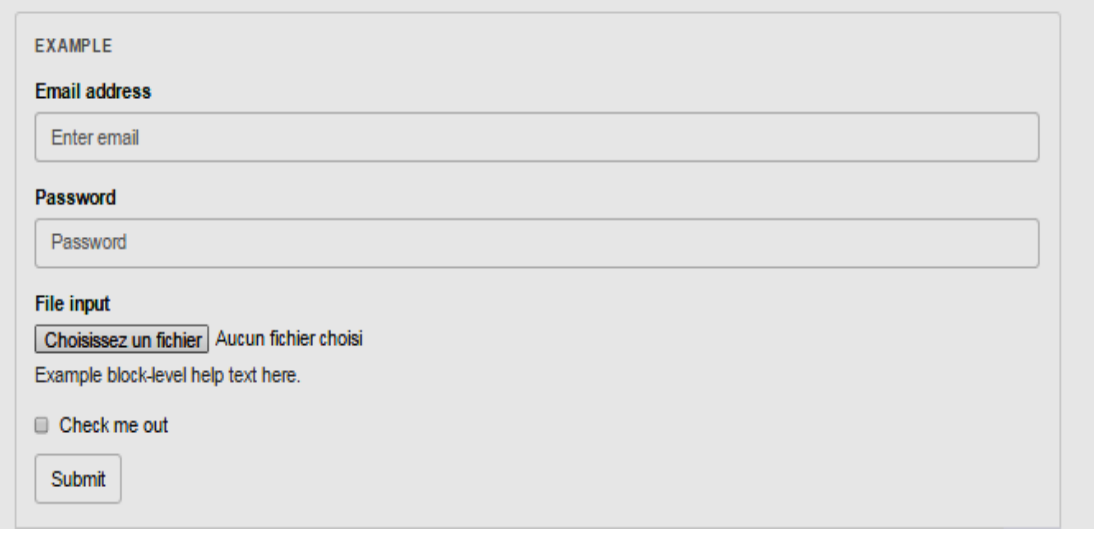
Avec un petit écran, l'utilisateur pourra utiliser une barre de défilement horizontale pour afficher tout le tableau, uniquement si cela est nécessaire.

VI. Mise en forme des formulaires

1. Structure de base

Pour une mise en forme optimale, Bootstrap préconise de placer les éléments du formulaire dans des boites utilisant la classe dédiée **.form-group**.

La classe **.form-contrôle** quant à elle permet au champ du formulaire d'occuper 100% de son conteneur.



```
<form>
  <div class="form-group">
    <label for="exampleInputEmail1">Email address</label>
    <input type="email" class="form-control" id="exampleInputEmail1" placeholder="Enter email">
  </div>
  <div class="form-group">
    <label for="exampleInputPassword1">Password</label>
    <input type="password" class="form-control" id="exampleInputPassword1" placeholder="Password">
  </div>
  <div class="form-group">
    <label for="exampleInputFile">File input</label>
    <input type="file" id="exampleInputFile">
    <p class="help-block">Example block-level help text here.</p>
  </div>
  <div class="checkbox">
    <label>
      <input type="checkbox"> Check me out
    </label>
  </div>
  <button type="submit" class="btn btn-default">Submit</button>
</form>
```

2. Affichage linéaire du formulaire

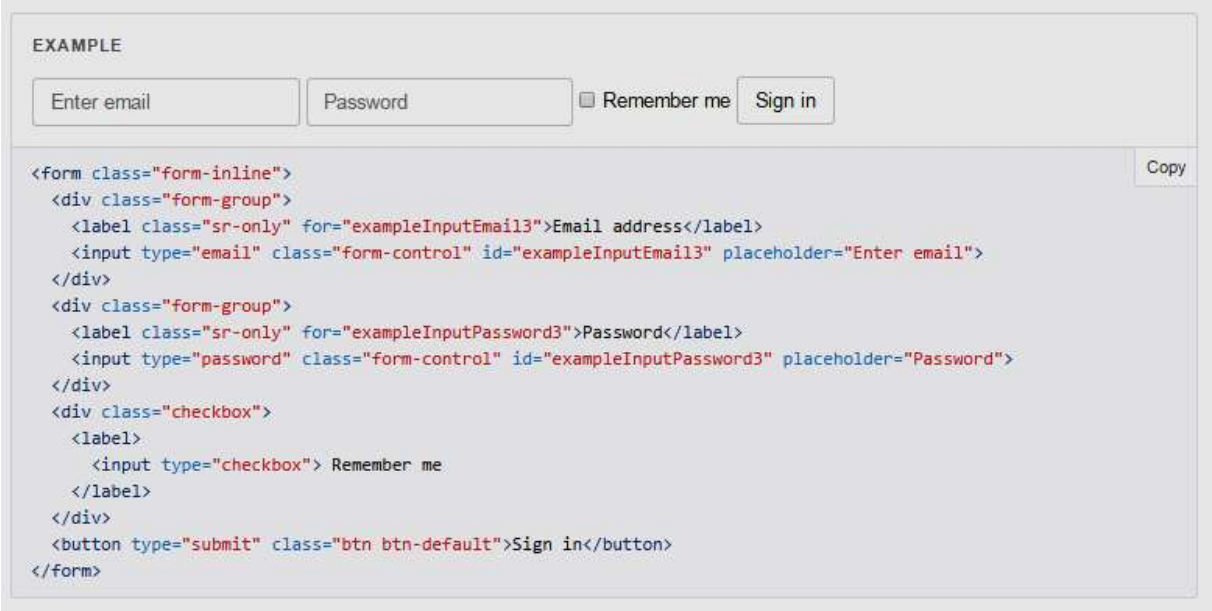
La classe **.form-inline** appliquée à l'élément <form> permet d'avoir les champs du formulaire en ligne, c'est-à-dire les uns à la suite des autres et non plus les uns sous les autres.

EXAMPLE

Name Jane Doe Email jane.doe@example.com Send invitation

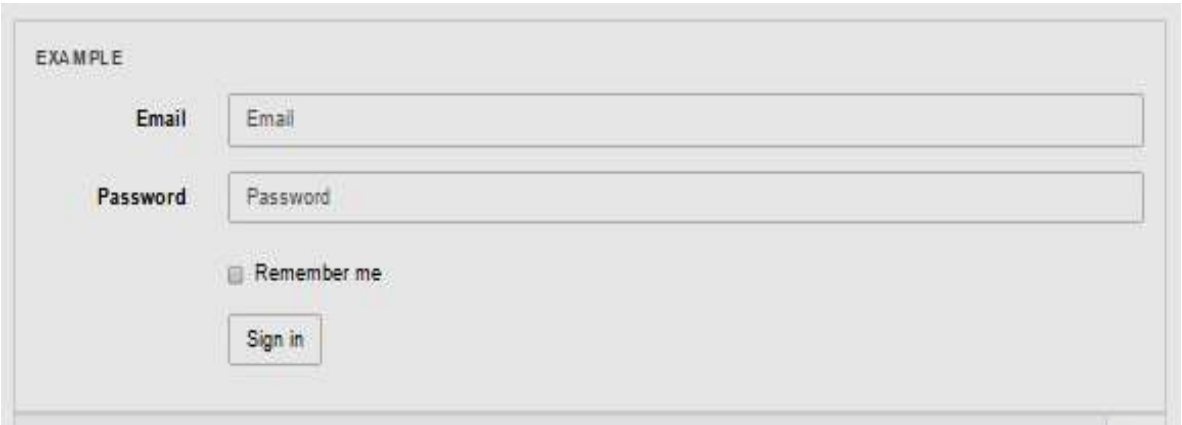
```
<form class="form-inline">
  <div class="form-group">
    <label for="exampleInputName2">Name</label>
    <input type="text" class="form-control" id="exampleInputName2" placeholder="Jane Doe">
  </div>
  <div class="form-group">
    <label for="exampleInputEmail2">Email</label>
    <input type="email" class="form-control" id="exampleInputEmail2" placeholder="jane.doe@example.com">
  </div>
  <button type="submit" class="btn btn-default">Send invitation</button>
</form>
```

La deuxième particularité de cet exemple de formulaire, c'est l'utilisation de la classe **.sr-only** dans les éléments <label> qui permet de ne pas afficher les éléments qui utilisent cette classe.



3. Affichage Vertical du formulaire

Bootstrap permet d'aligner les étiquettes et l'élément du formulaire verticalement avec l'utilisation de la classe **.form-horizontal** dans l'élément <form>.

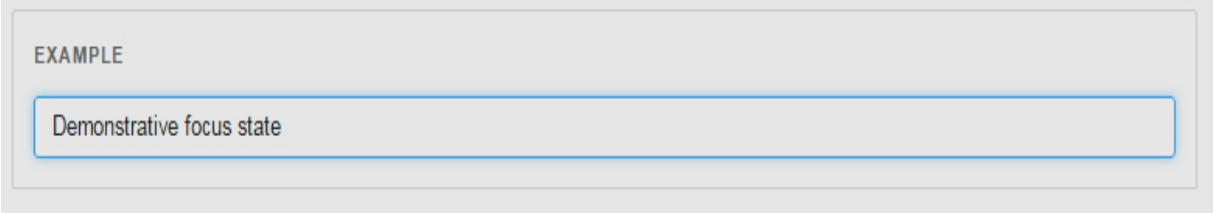


```
<form class="form-horizontal">
  <div class="form-group">
    <label for="inputEmail3" class="col-sm-2 control-label">Email</label>
    <div class="col-sm-10">
      <input type="email" class="form-control" id="inputEmail3" placeholder="Email">
    </div>
  </div>
  <div class="form-group">
    <label for="inputPassword3" class="col-sm-2 control-label">Password</label>
    <div class="col-sm-10">
      <input type="password" class="form-control" id="inputPassword3" placeholder="Password">
    </div>
  </div>
  <div class="form-group">
    <div class="col-sm-offset-2 col-sm-10">
      <div class="checkbox">
        <label>
          <input type="checkbox"> Remember me
        </label>
      </div>
    </div>
  </div>
  <div class="form-group">
    <div class="col-sm-offset-2 col-sm-10">
      <button type="submit" class="btn btn-default">Sign in</button>
    </div>
  </div>
</form>
```

Cet affichage est basé sur la grille adaptative que nous verons par la suite. C'est donc a vous d'utiliser les classes voulues dans les éléments , <label> et <input>.

4. Les champs de saisie

Quand un champ est actif, Bootstrap le met nativement en évidence avec un contour blue ombré.

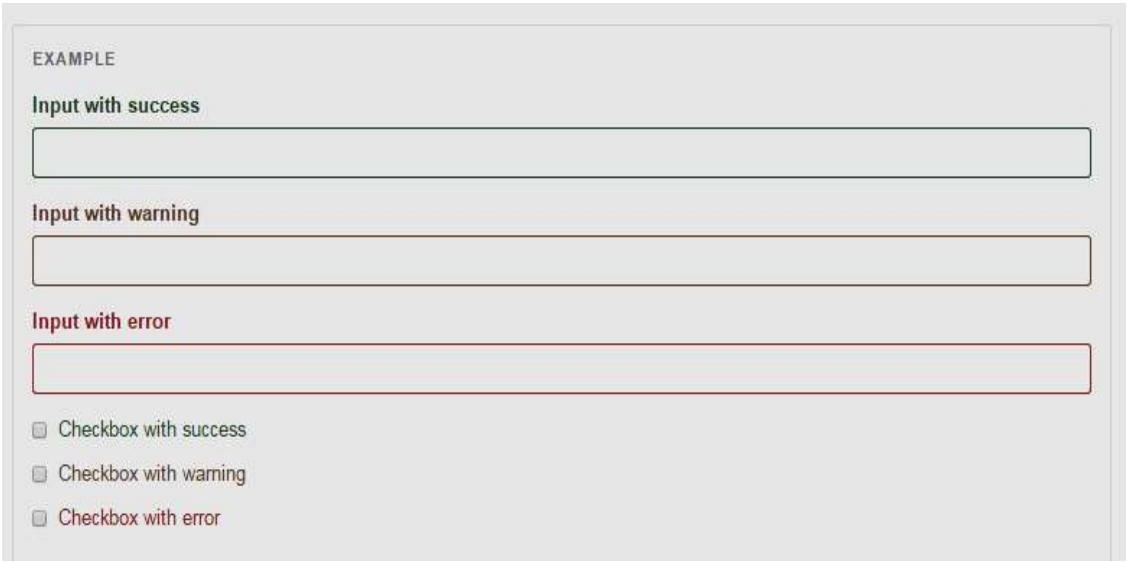


Pour le désactiver, il suffit d'utiliser l'attribut **desabled.**



Vous pouvez utiliser trois classes prédéfinies pour mettre en couleur les champs de saisie :

- .has-seccess qui utilise une couleur verte ;
- .has-warning qui utilise une couleur orange ;



- .has-error qui utilise une couleur rouge.

Avec le Bootstrap, vous pouvez également indiquer une aide pour les champs du formulaire, pour donner des consignes aux utilisateurs du formulaire. Chaque aide sera affichée sous les champs. Pour ce faire, vous pouvez utiliser un élément avec la classe **.help-block**.

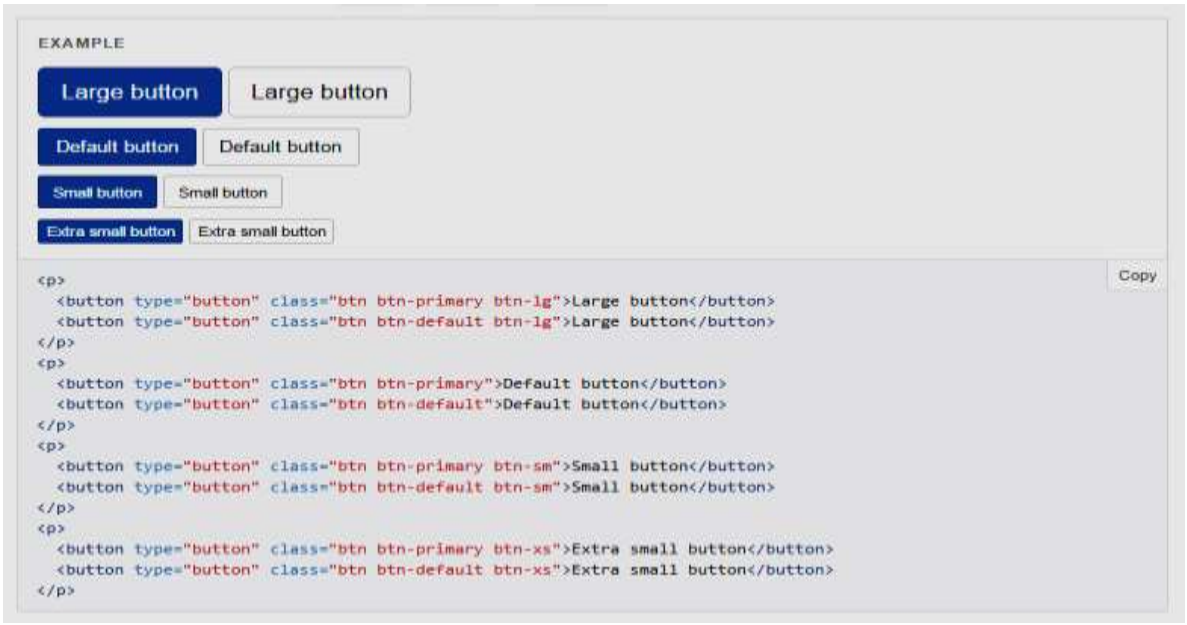
5. Les boutons

Bootstrap vous propose plusieurs classes pour colorer vos boutons. Vous pouver donner un sens à vos boutons, de sorte que vos utilisateurs aient une meilleure expérience dans vos interfaces.



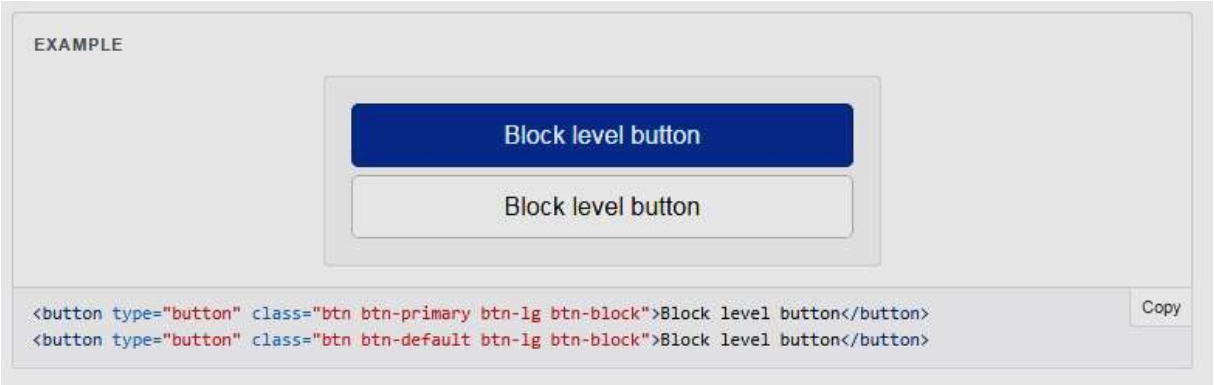
➢ Largeur des boutons

Vous pouvez gérer la largeur des boutons à l'aide de trois classes :.btn-lg, .btn-sm, ou .btn-xs.



➢ Les boutons en bloc

La classe **.btn-block** va vous permettre de créer des gros boutons en affichage bloc.



➢ Les boutons de formulaire

Pour la mise en forme des boutons d'envoi ou d'annulation, utilisez les classes **.btn** et **.btn-default** dans l'élément <input>.

➢ Désactiver un bouton

Nous l'avons déjà vu avec les formulaires, l'attribut **disabled**



permet de rendre inactif un bouton avec une mise en forme spéciale.

Cet attribut peut également être appliqué à un bouton de type lien :



VII. Mise en forme de Menu (Horizontal et Vertical)

1. La navigation verticale

Les classes **.nav** et **.nav-pills** vont vous permettre d'avoir une navigation par onglets/boutons. Pour avoir une navigation verticale vous devez associer la classe **.nav-stacked**.

Comme les boutons de la liste <ul> occupent 100% de la largeur disponible de l'élément parent, pour éviter d'avoir des boutons très/trop larges, nous plaçons la liste <ul> dans une boite ou autre ayant une largeur fixe (width : 200px).



2. La navigation Horizontale

La barre de navigation est une boite contenant la classe **.navbar**. La mise en forme par défaut de cette barre de navigation est appliquée avec la classe **.navbar-default**. Ensuite il y'a les classes **.collapse et .navbar-collapse** qui permettent la mise en forme du contenu de la barre de navigation. Les liens <a> sont

placés dans l'élément <li> d'une liste <ul>. L'élément <ul> utilise les deux classes de mise en forme **.nav** et **.navbar-nav.**

```
<navclass="navbarnavbar-default">
<divclass="collapse navbar-collapse"id="bs-example-navbar-collapse-
1">
<ulclass="navnavbar-nav">
<liclass="active"><ahref="#">Link <spanclass="sr-
only">(current)</span></a></li>
<li><ahref="#">Link</a></li>
<liclass="dropdown">
<ahref="#"class="dropdown-toggle"data-
toggle="dropdown"role="button"aria-expanded="false">Dropdown
<spanclass="caret"></span></a>
        <ulclass="dropdown-menu"role="menu">
                <li><ahref="#">Action</a></li>
                <li><ahref="#">Another action</a></li>
                <li><ahref="#">Something else here</a></li>
                <liclass="divider"></li>
                <li><ahref="#">Separated link</a></li>
                <liclass="divider"></li>
                <li><ahref="#">One more separated link</a></li>
        </ul>
</li>
</ul>
</div>
</nav>
```

VIII. Les composants JavaScript du Bootstrap

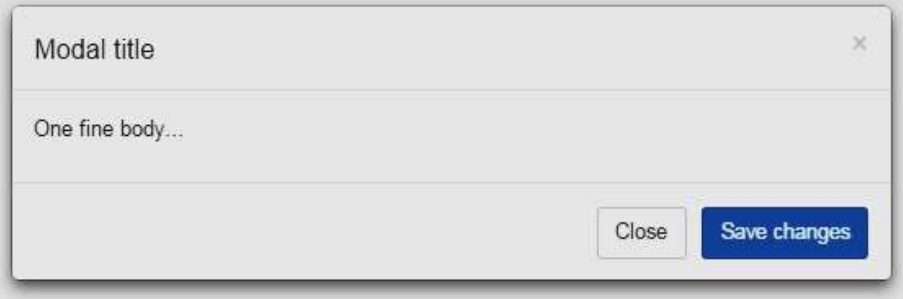
Tous les plugins JavaSript du bootstrap ont leur code fonctionnel dans le fichier **bootstrap.min.js** ou (**bootstrap.js**), mais attention n'utilisez pas les deux fichiers et n'oubliez pas d'importer la librairie Jquery dans votre application.

1. Les fenêtres modales

Vous pouvez faire apparaitre une fenêtre de contenu au clic d'un bouton par exemple. Cela va permettre d'afficher des informations complémentaires sur un produit par exemple.

Pour cela vous allez utiliser une fenêtre modale, elle est composée de trois parties : l'en-tête, le corps et le pied de page. La boite de la racine contient tous les attributs nécessaires à son bon fonctionnement :

EXAMPLE



Copy

```
<div class="modal fade">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <button type="button" class="close" data-dismiss="modal" aria-label="Close"><span aria-
hidden="true">&times;</span></button>
        <h4 class="modal-title">Modal title</h4>
      </div>
      <div class="modal-body">
        <p>One fine body&hellip;</p>
      </div>
      <div class="modal-footer">
        <button type="button" class="btn btn-default" data-dismiss="modal">Close</button>
        <button type="button" class="btn btn-primary">Save changes</button>
      </div>
    </div><!-- /.modal-content -->
  </div><!-- /.modal-dialog -->
</div><!-- /.modal -->
```

2. Les alertes

Les alertes sont plutôt faites pour les interfaces d'administration des CMS, mais vous pouvez les utiliser pour mettre en évidence des propos important dans votre application web.



3. Le Carrousel



Copy

```
<div id="carousel-example-generic" class="carousel slide" data-ride="carousel">
  <!-- Indicators -->
  <ol class="carousel-indicators">
    <li data-target="#carousel-example-generic" data-slide-to="0" class="active"></li>
    <li data-target="#carousel-example-generic" data-slide-to="1"></li>
    <li data-target="#carousel-example-generic" data-slide-to="2"></li>
  </ol>

  <!-- Wrapper for slides -->
  <div class="carousel-inner" role="listbox">
    <div class="item active">
      <img src="..." alt="...">
      <div class="carousel-caption">
        ...
      </div>
    </div>
    <div class="item">
      <img src="..." alt="...">
      <div class="carousel-caption">
        ...
      </div>
    </div>
    ...
  </div>

  <!-- Controls -->
  <a class="left carousel-control" href="#carousel-example-generic" role="button" data-slide="prev">
    <span class="glyphicon glyphicon-chevron-left" aria-hidden="true"></span>
    <span class="sr-only">Previous</span>
  </a>
  <a class="right carousel-control" href="#carousel-example-generic" role="button" data-slide="next">
    <span class="glyphicon glyphicon-chevron-right" aria-hidden="true"></span>
    <span class="sr-only">Next</span>
  </a>
</div>
```

PARTIE 4 : JAVASCRIPT – PROGRAMMATION (Dynamiser la page)

I. Historique du JavaScript

En 1995, Brendan Eich travaille chez Netscape Communication Corporation, la société qui éditait le célèbre navigateur Netscape Navigator, alors principal concurrent d'Internet Explorer. Brendan développe le LiveScript, un langage de script qui s'inspire du langage Java, et qui est destiné à être installé sur les serveurs développés par Netscape. Netscape se met à développer une version client du LiveScript, qui sera renommée JavaScript en hommage au langage Java créé par la société Sun Microsystems. En effet, à cette époque, le langage Java était de plus en plus populaire, et appeler le LiveScript JavaScript était une manière de faire de la publicité, et au Java, et au JavaScript lui-même. Mais attention, au final, ces deux langages sont radicalement différents ! N'allez pas confondre le Java et le Javascript car ces deux langages n'ont clairement pas le même fonctionnement.

II. Les bases du langage JavaScript

1. Le concept de variable

Une variable est un objet repéré par son nom, pouvant contenir des données, qui pourront être modifiées lors de l'exécution du programme.

La déclaration des variables se fait en précédant la variable du mot clé *var* qui permet d'indiquer de façon rigoureuse qu'il s'agit d'une variable : *Exemple* : var chaine= "bonjour"

```
<SCRIPT language="Javascript">
        var MaVariable;
        var MaVariable2 = 3;
</SCRIPT>
```

2. Les types de variables

Les variables JavaScript peuvent contenir de nombreux **types de données** : nombres, chaînes, objets et plus. Voici quelques types de variables usuelles :

a. Des nombres : entiers ou à virgules

JavaScript n'a qu'un seul type de nombres. Les nombres peuvent être écrits avec ou sans décimales

Exemple

```
<SCRIPT language="Javascript">
        var x1 = 34.00 ;
        var x2 = 34 ;
</SCRIPT>
```

b. Des chaînes de caractères (string) : une suite de caractères

Une chaîne (ou une chaîne de texte) est une série de caractères comme "Bonjour les amis". Les chaînes sont écrites avec des guillemets. Vous pouvez utiliser des guillemets simples ou doubles :

```
<SCRIPT language="Javascript">
        var nom = "Langage" ;
        var prenom = 'JavaScript' ;
</SCRIPT>
```

Vous pouvez utiliser des guillemets dans une chaîne, tant qu'ils ne correspondent pas aux guillemets entourant la chaîne :

```
<SCRIPT language="Javascript">
        var hello = "Je m'appelle 'HELLO' " ;
        let hello2 = 'Je me nomme "HELLO" ' ;
</SCRIPT>
```

Remarque : **var** est utilisé pour déclarer une variable globale alors que **let** est utilisé pour déclarer une variable dont la portée est limitée à un bloc. Ainsi, vous pouvez déclarer une variable à l'aide de let dans un bloc if mais sa portée est limitée à ce bloc. Vous pouvez même déclarer plusieurs variables portant le même nom à l'intérieur et à l'extérieur d'un bloc à l'aide du mot-clé **let**.

c. Des booléens : des variables permettant de vérifier une condition.

Les booléens peuvent prendre deux états :

- true : si le résultat est vrai ;
- false : lors d'un résultat faux.

d. Les objets

Les objets sont déclarés en utilisant des accolades. Ils sont destinés à stocker plusieurs types de valeurs et des méthodes (fonctions).

Exemple

```
<SCRIPT language="Javascript">
    var formateur= {nom :"TOTO", sexe : 'M', age :22} ;
    alert(formateur['nom']) ;
    alert(formateur['sexe']) ;
</SCRIPT>
```

3. Conversion des variables

Même si Javascript gère de façon transparente les changements de type des variables, il est parfois nécessaire de forcer la conversion du type. Ainsi Javascript fournit deux fonctions natives permettant de convertir le type des variables passées en paramètre :

- parseInt() permet de convertir une variable en nombre
- parseFloat() permet de convertir une variable en nombre décimal

parseInt()

La fonction *parseInt()* permet de convertir une variable passée en paramètre. La syntaxe est : parseInt(ma_var_chaine);

Pour que la fonction *parseInt()* retourne un entier, la chaine passée en paramètre doit commencer par des caractère valides c'est-à-dire les chiffres [0-9] ou le préfixe hexadécimal *0x*, et/ou les caractères *+*, *-*, *E* et *e*. Dans le cas contraire la fonction *parseInt()* retournera la valeur *NaN* (*Not a Number*).

Exemple :

var a = "123";

var b = "456";

Selon que l'on utilise la fonction *parseInt()* ou non, l'utilisation de l'opérateur + avec ces deux variables donnera un résultat différent :

a+b ; // Affiche 123456

parseInt(a)+parseInt(b); // Affiche 579

parseFloat()

La fonction *parseFloat()* est une fonction du noyau Javascript permettant de convertir une variable passée en paramètre en nombre en virgule flottante (nombre avec une partie décimale). La syntaxe de la fonction *parseFloat()* est la suivante : parseFloat(chaine);

4. Les opérateurs

a. Affectation (=) et autres

L'opérateur d'affectation (=) attribue une valeur à une variable.

Opérateur d'affectation	Exemple	Exemple équivalent
=	i = 20	i = 20
+=	i += 20	i = i + 20
-=	i -= 20	i = i - 20

*=	i *= 20	i = i * 20
/=	i /= 20	i = i / 20
%=	i %= 20	i = i % 20

b. Opérateurs arithmétiques

Opérateur	Description	Exemple
Addition +	- ajoute des nombres - fait la concaténation des chaines de caractères	var i = 0 ; i = i + 20 var Langage = "Java"+"Script" ; var age="J'ai" + 21 + "ans" ; var titre = 007+" James Bond" ;
Soustraction -	soustrait des nombres	var i = 0 ; i = i - 20
Multiplication *	multiplie les nombres	var i = 0 ; i = i * 20
Division /	divise les nombres	var i = 0 ; i = i / 20
Modulo %	Reste de la division entière	var i = 21 ; i = i % 2
Incrément ++	ajoute la valeur au nombre	var i = 0 ; i++
Décrément --	soustrait la valeur au nombre	var i = 0 ; i--

c. Les opérateurs de comparaison

Opérateur	Description	Exemple
==	Valeur égale	5 == 5
===	Valeur égale et type égal	5 === 5.0
!=	Valeur différente	10.0 != 30

Opérateur	Description	Exemple
!==	Valeur différente et type différent	10.0 != 10
>	Supérieur à	15 > 3.14
>=	Supérieur ou égal à	15 >= 3.14
<	Inférieur à	2021 < 2020
<=	Inférieur ou égale à	2021 <= 2020

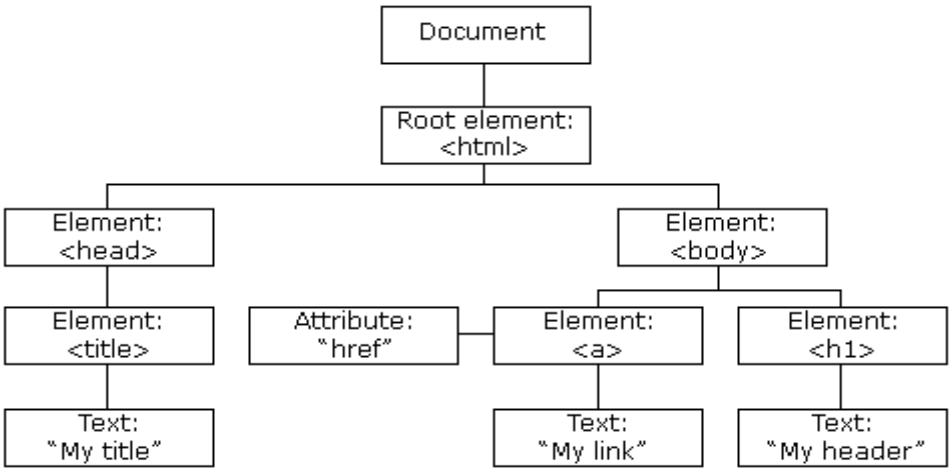
d. Les opérateurs logiques

Opérateur	Description	Exemple
&&	Et logique	(age==18) && (sexe="f")
||	Ou logique	(age==18) || (sexe="f")
!	Non logique	var refus = !false

III. Le DOM HTML (Document Object Model)

Lorsqu'une page Web est chargée, le navigateur crée un **D**ocument **O**bjet **M**odèle de la page.

Le modèle **HTML DOM** est construit comme une arborescence d' **objets** :



Avec le modèle objet, JavaScript obtient toute la puissance dont il a besoin pour créer du HTML dynamique :

- JavaScript peut modifier tous les éléments HTML de la page
- JavaScript peut modifier tous les attributs HTML de la page
- JavaScript peut changer tous les styles CSS de la page
- JavaScript peut supprimer les éléments et attributs HTML existants
- JavaScript peut ajouter de nouveaux éléments et attributs HTML
- JavaScript peut réagir à tous les événements HTML existants de la page
- JavaScript peut créer de nouveaux événements HTML dans la page

1. Qu'est-ce que le DOM HTML ?

Le DOM HTML est un modèle d'objet standard et **une interface de programmation** pour HTML. Il définit :

- Les éléments HTML comme **objets**
- Les **propriétés** de tous les éléments HTML
- Les **méthodes** pour accéder à tous les éléments HTML
- Les **événements** pour tous les éléments HTML

En d'autres termes : **le DOM HTML est une norme pour savoir comment obtenir, modifier, ajouter ou supprimer des éléments HTML.**

2. Méthodes HTML DOM

Les méthodes HTML DOM sont des **actions que** vous pouvez effectuer (sur des éléments HTML).

Les propriétés HTML DOM sont des **valeurs** (d'éléments HTML) que vous pouvez définir ou modifier.

Exemple

L'exemple suivant modifie le contenu (innerHTML) de l'élément <p> avec id="demo":

```
<html>
    <head>
        <meta charset="utf-8" />
        <title>title</title>
    </head>
<body>
    <p id="demo"></p>
    <script> document.getElementById("demo").innerHTML =
    "Hello World!"; </script>
</body>
</html>
```

3. La méthode getElementById

Le moyen le plus courant d'accéder à un élément HTML est d'utiliser le id de l'élément.

Dans l'exemple ci-dessus, la getElementById méthode utilisée id="demo"pour trouver l'élément.

4. La propriété innerHTML

Le moyen le plus simple d'obtenir le contenu d'un élément consiste à utiliser la propriété innerHTML. Elle est utile pour obtenir ou remplacer le contenu des éléments HTML. La propriété innerHTML peut être utilisée pour obtenir ou modifier n'importe quel élément HTML, y compris <html>et <body>.

5. Recherche d'éléments HTML

Méthode (fonction)	**Description**
document.getElementById(*id*)	Trouve un élément par son id
document.getElementsByTagName(*name*) document.getElementsByName(*name*)	Trouve un élément par son tag name ou name
document.getElementsByClassName(*name*)	Trouve un élément par la class (nom de la class)

6. Modification des éléments HTML

Propriété	**Description**
element.innerHTML = *new html content*	Change innerHTML d'un élément
element.attribute = new value	Affecte une valeur à l'attribut d'un élément HTML

element.style.*property* = *new style*	Change le style d'un élément HTML
element.setAttribute*(attribute, value)*	Change la valeur de l'attribut d'un élément HTML

7. Ajout et suppression d'éléments

Fonction	**Description**
document.createElement(*element*)	Crée un élément HTML
document.removeChild(*element*)	Supprime un élément HTML
document.appendChild(*element*)	Ajoute un élément HTML
document.replaceChild(*new, old*)	Remplace un élément HTML
document.write(*text*)	Ecrire dans le flux de sortie HTML

8. Ajout de gestionnaires d'événements

Fonction	**Description**
document.getElementById(*id*).onclick = function(){*code*}	Ajout de code de gestionnaire d'événements à un événement onclick

9. Quelques possibilités d'affichage JavaScript

JavaScript peut "afficher" les données de différentes manières :

- Ecrire dans un élément HTML, en utilisant **innerHTML**.
- Ecriture dans la sortie HTML en utilisant **document.write()**.
- Ecrire dans une boîte d'alerte, en utilisant **window.alert()**.
- Ecriture dans la console du navigateur, en utilisant **console.log()**.

a. Utilisation de innerHTML

Pour accéder à un élément HTML, JavaScript peut utiliser la méthode **document.getElementById(id)**. L'attribut id définit l'élément HTML. La propriété innerHTML définit le contenu HTML :

Exemple

```
<!doctype html>
<html>
	<head>
		<meta charset="utf-8" />
		<title>title</title>
	</head>
<body>
		<p id="para1"></p>
		<script>
			document.getElementById("para1").innerHTML = "Ce message est écrit par un script JavaScript" ;
		</script>
</body>
</html>
```

b. Utilisation de document.write ()

À des fins de test, il est pratique d'utiliser document.write():

Exemple

```
<!doctype html>
<html>
	<head>
		<meta charset="utf-8" />
		<title>Document.write</title>
	</head>
<body>
```

```
        <script>
            document.write("Ce message est écrit par un script JavaScript") ;
        </script>
</body>
</html>
```

c. Utilisation de window.alert ()

Vous pouvez utiliser une boîte d'alerte pour afficher des données :

Exemple

```
<!doctype html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Window.alert</title>
    </head>
<body>
        <script>
            window.alert("Hello world ! Ce message est écrit par un script JavaScript") ;
        </script>
</body>
</html>
```

d. Utilisation de console.log ()

À des fins de débogage, vous pouvez appeler la méthode console.log() dans le navigateur pour afficher les données.

Exemple

```
<!doctype html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Console.log</title>
```

```
    </head>
<body>
        <script>
            console.log("Hello world ! Ce message est écrit par un script JavaScript") ;
        </script>
</body>
</html>
```

e. Impression JavaScript

En JavaScript vous pouvez appeler la méthode window.print() dans le navigateur pour imprimer le contenu de la fenêtre actuelle.

Exemple

```
<!doctype html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>title</title>
    </head>
<body>
    <p id="para1"></p>
    <script>
        document.getElementById("para1").innerHTML = "Ce message est écrit par un script JavaScript" ;
    </script>
    <button onclick="window.print()">Print this page</button>
</body>
</html>
```

IV. Les boites de dialogues (alert, confirm, prompt)

JavaScript prend en charge trois (03) types de boîtes de dialogue. Ces boîtes de dialogue peuvent être utilisées pour

déclencher et alerter, ou pour obtenir une confirmation sur n'importe quelle entrée ou pour avoir une sorte d'entrée des utilisateurs. Ici, nous allons discuter de chaque boîte de dialogue une par une.

1. Alert()

Une boîte de dialogue d'alerte est principalement utilisée pour donner un message d'avertissement aux utilisateurs.

Syntaxe :

window.alert("*message d'alerte*");

Exemple

html :	Script : javascript.js

<!doctype html>
<html>
<head>
<meta charset="utf-8" />
<title>Boite ALERT </title>
<script src="javascript.js">
</script> </head>
<body onload="bonjour() ;">
 Ma boite avec javascript.
<button type="button" onclick=" bonjour ()">Try it</button>
</body>
</html>
	function bonjour() {
	alert('Hello, world!');
	}

2. Confirm()

Une boîte de dialogue de confirmation est principalement utilisée pour obtenir le consentement de l'utilisateur sur n'importe quelle

option. Il affiche une boîte de dialogue avec deux boutons : **OK** et **Annuler**.

Syntaxe :

window.confirm("*Texte de confirmation*");

Exemple

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Boite de confirmation </h2>
<button onclick="myFunction()">Try it</button>
<p id="demo"></p>

<script>
function myFunction() {
    var txt;
    if (confirm("Press a button!")) {
        txt = "Tu as appuyé sur OK!";
    } else {    txt = " Tu as appuyé sur Cancel!";  }
    document.getElementById("demo").innerHTML = txt;
}
</script>
</body>
</html>
```

3. Prompt()

La boîte de dialogue d'invite est très utile lorsque vous souhaitez afficher une zone de texte pour obtenir une entrée utilisateur. Ainsi, il vous permet d'interagir avec l'utilisateur. L'utilisateur doit remplir le champ puis cliquer sur OK.

Syntaxe :

window.prompt("*Texte indicatif*"[, *valeur par défaut*]);

Exemple

```
<!DOCTYPE html>
<html>
<body>
<h2>JavaScript Prompt</h2>
<button onclick="myFunction()">Try it</button>
<p id="demo"></p>
<script>
function myFunction() {
    var txt;
    var person = prompt("Donner votre nom:", "M. AGBOKA");
    if (person == null || person == "") {
        txt = "User cancelled the prompt.";
    } else {
        txt = "Hello " + person + "! How are you today?";
    }
    document.getElementById("demo").innerHTML = txt;
}
```

```
</script>
</body> </html>
```

V. Les structures conditionnelles/alternatives (IF..ELSE, SWITCH)

1. Structure IF

La structure IF se présente comme suit :

```
if (condition) {
        « instructions »
}
```

OU

```
if (condition) {
        « instructions »
} else {
        « instructions »
}
```

2. Structure SWITCH

Utilisez l'instruction switch pour sélectionner l'un des nombreux blocs de code à exécuter. La syntaxe se présente comme suit :

```
Switch (expression) {
        case n:
                « instructions »
                break;
        case n:
                « instructions »
                break;
        default:
                « Instructions »
}
```

VI. Les structures itératives (WHILE, DO…WHILE, FOR, FOR…IN)

JavaScript prend en charge différents types de boucles :

- while - boucle à travers un bloc de code alors qu'une condition spécifiée est vraie
- do / while - boucle également à travers un bloc de code alors qu'une condition spécifiée est vraie
- for - parcourt un bloc de code plusieurs fois
- for / in - parcourt les propriétés d'un objet

1. La structure while

Les boucles peuvent exécuter un bloc de code tant qu'une condition spécifiée est vraie. La syntaxe est :

while (*condition*) {
« instrutions »
}

2. La structure Do .. while

La boucle do / while est une variante de la boucle while. Cette boucle exécutera le bloc de code une fois, avant de vérifier si la condition est vraie, puis elle répétera la boucle tant que la condition est vraie.

3. La structure For

La boucle for a la syntaxe suivante:

for (*statement 1*; *statement 2*; *statement 3*) {
« Instructions »
}

L'instruction 1 est exécutée (une fois) avant l'exécution du bloc de code.

L'instruction 2 définit la condition d'exécution du bloc de code.

L'instruction 3 est exécutée (à chaque fois) après l'exécution du bloc de code.

4. La structure For .. In

L'instruction JavaScript for / in parcourt les propriétés d'un objet :

Exemple

```
var person = {nom:"John", prenoms:"Doe", age:25};
var text = "";
var x;
for (x in person) {
        text += person[x];
}
```

VII. Les tableaux

1. Tableau (type élémentaire)

Les tableaux sont déclarés pour stocker plusieurs valeurs de même type. L'indice du 1er élément du tableau est 0 (zéro).

Syntaxe :

Var nomtableau [« liste des valeurs séparées par des virgules »]

Ou

var nom_tableau = new Array(« liste des valeurs séparée par des virgules »);

Exemple

```
var cars = ["Toyota", "Mazda", "Honda", "Saab", "Volvo", "BMW"];
Alert(cars[0]) ;
var fruits = new Array( "pomme", "orange", "mangue" );
```

2. Quelques méthodes tableau

Le tableau en javascript dispose de plus d'une vingtaine de fonctions (méthodes) dont voici quelques-unes.

Méthodes ou fonction tableau	**exemples**
Length Compte le nombre d'éléments du tableau	cars.length ;
Push() Ajoute un nouvel élément à la fin au tableau	cars.push("Peugeot") ;
Pop() Supprime le dernier élément du tableau	cars.pop()
Unshift() Ajoute un nouvel élément au début du tableau	cars.unshift("Ferrari")
Shift() Supprime le 1er élément du tableau	cars.shift() ;
Sort() Trie les éléments du tableau	cars.sort()

VIII. Les fonctions

Les fonctions JavaScript sont **définies** avec le mot-clé **function** .

Vous pouvez utiliser une **déclaration de** fonction ou une **expression de** fonction.

1. Déclaration de fonction

Syntaxe :	Exemple

function *functionName*(*parametres*) {
« *instructions* »
}
	function f_sum(a, b) {
	return a * b;
	}

2. Expressions de fonction

Une fonction JavaScript peut également être définie à l'aide d'une **expression**.

Syntaxe :

Var *variableName* = function (*parametres*) {
    « *instructions* »
}

Exemple

```
<!DOCTYPE html>
<html>
<body>
    <p>Une fonction stockée dans une variable :</p>
    <p id="demo"></p>
    <script>
    var som = function (a, b) {return a * b};
    document.getElementById("demo").innerHTML = som(2,50);
    </script>
</body>
</html>
```

IX. Les évènements

Les événements sont des actions de l'utilisateur, qui vont pouvoir donner lieu à une interactivité. L'événement par excellence est le clic de souris, car c'est le seul que le HTML gère. Grâce au JavaScript il est possible d'associer des fonctions, des méthodes à des événements tels que le passage de la souris au-dessus d'une zone, le changement d'une valeur, ...

Ce sont les gestionnaires d'événements qui permettent d'associer une action à un événement. La syntaxe d'un gestionnaire d'événement est la suivante :

on*Evenement*="Action_Javascript_ou_Fonction();"

Lorsqu'il est utilisé dans un lien hypertexte, par exemple, la syntaxe sera la suivante :

<A href="URL" "on*Evenement*='Action_Javascript_ou_Fonction();'">
Lien

Evénement	**Description**
Abort (onAbort)	Cet événement a lieu lorsque l'utilisateur interrompt le chargement de l'image
Blur (onBlur)	Se produit lorsque l'élément perd le focus, c'est-à-dire que l'utilisateur clique hors de cet élément, celui-ci n'est alors plus sélectionné comme étant l'élément actif.
Change (onChange)	Se produit lorsque l'utilisateur modifie le contenu d'un champ de données.
Click (onClick)	Se produit lorsque l'utilisateur clique sur l'élément associé à l'événement.
dblclick (onDblclick)	Se produit lorsque l'utilisateur double-clique sur l'élément associé à l'événement (un lien hypertexte ou un élément de formulaire). Cet événement n'est supporté que par les versions de Javascript 1.2 et supérieures

Evénement	Description
dragdrop (onDragdrop)	Se produit lorsque l'utilisateur effectue un *glisser-déposer* sur la fenêtre du navigateur. Cet événement n'est supporté que par les versions de Javascript 1.2 et supérieures
error (onError)	Se déclenche lorsqu'une erreur apparaît durant le chargement de la page. Cet événement fait partie du Javascript 1.1.
Focus (onFocus)	Se produit lorsque l'utilisateur donne le focus à un élément, c'est-à-dire que cet élément est sélectionné comme étant l'élément actif
keydown (onKeydown)	Se produit lorsque l'utilisateur appuie sur une touche de son clavier. Cet événement n'est supporté que par les versions de Javascript 1.2 et supérieures
keypress (onKeypress)	Se produit lorsque l'utilisateur maintient une touche de son clavier enfoncée. Cet événement n'est supporté que par les versions de Javascript 1.2 et supérieures
keyup (onKeypress)	Se produit lorsque l'utilisateur relâche une touche de son clavier préalablement enfoncée. Cet événement n'est supporté que par les versions de Javascript 1.2 et supérieures
Load (onLoad)	Se produit lorsque le navigateur de l'utilisateur charge la page en cours
MouseOver (onMouseOver)	Se produit lorsque l'utilisateur positionne le curseur de la souris au-dessus d'un élément

Evénement	Description
MouseOut (onMouseOut)	Se produit lorsque le curseur de la souris quitte un élément. Cet événement fait partie du Javascript 1.1.
Reset (onReset)	Se produit lorsque l'utilisateur efface les données d'un formulaire à l'aide du bouton Reset.
Resize (onResize)	Se produit lorsque l'utilisateur redimensionne la fenêtre du navigateur
Select (onSelect)	Se produit lorsque l'utilisateur sélectionne un texte (ou une partie d'un texte) dans un champ de type "text" ou "textarea"
Submit (onSubmit)	Se produit lorsque l'utilisateur clique sur le bouton de soumission d'un formulaire (le bouton qui permet d'envoyer le formulaire)
Unload (onUnload)	Se produit lorsque le navigateur de l'utilisateur quitte la page en cours

ANNEXE

I. La liste complète des balises HTML5

Balise	Description
<!– … –>	Définit un commentaire
<!DOCTYPE>	Définit le type du document
<a>	Définit un lien
<abbr>	Définit une abréviation
<address>	Définit une adresse
<area>	Définit une zone à l'intérieur d'une image
<article>NOUVEAU	Définit un article
<aside>NOUVEAU	Définit une partie latérale au contenu
<audio>NOUVEAU	Définit un fichier audio
<b>	Texte en gras
<base>	Définit une URL de base pour tous les liens de la page
<bdo>	Définit la direction du texte
<blockquote>	Définit une longue citation
<body>	Définit le corps de la page
 	Saut de ligne
<button>	Définit un bouton cliquable
<canvas>NOUVEAU	Définit un graphique
<caption>	Légende du tableau
<cite>	Définit une citation

<code>	Mise en forme d'un texte en code informatique
<col>	Définit une colonne d'un tableau
<colgroup>	Définit un groupe de colonne pour un tableau
<command>NOUVEAU	Définit un bouton de commande
<datalist>NOUVEAU	Définit une liste d'options
<dd>	Définition d'un terme
<del>	Définit un texte supprimé
<details>NOUVEAU	Définit les détails d'un élément
<dfn>	Définition
	Définit un calque ou une section
<dl>	Liste de définition
<dt>	Définition d'un terme
	Mise en exergue d'un texte – italique
<embed>NOUVEAU	Définit un contenu extérieur (audio, vidéo …)
<fieldset>	Regroupe plusieurs éléments d'un formulaire
<figcaption>NOUVEAU	Légende d'un groupe d'élément multimédia
<figure>NOUVEAU	Définit un groupe d'élément multimédia
<footer>NOUVEAU	Définit le bas d'un section ou d'une page
<form>	Définit un formulaire
<h1> to <h6>	Définit un titre par degré importance de 1 à 6
<head>	Définit l'en-tête d'un document

<header>NOUVEAU	Définit le haut d'une section ou d'une page
<hgroup>NOUVEAU	Regroupe les informations du haut d'une page ou section
<hr>	Barre horizontale
<html>	Définit un document html
<i>	Texte en italique
<iframe>	Introduit un page html dans une frame
<img>	Définit une image
<input>	Définit un champ
<ins>	Définit un texte insérer
<keygen>NOUVEAU	Générateur de clé pour un formulaire
<kbd>	Raccourcis ou touche du clavier
<label>	Légende d'un groupe d'élément de formulaire
<legend>	Titre d'un groupe d'élément d'un formulaire
	Élément d'une liste
<link>	Définit les relations entre les documents
<map>	Définit une carte
<mark>NOUVEAU	Mise en valeur d'un mot ou d'un texte – Texte marqué
<math>NOUVEAU	Définit une formule mathématique
<menu>	Définit un menu en liste
<meta>	Définit des informations relatives au document
<meter>NOUVEAU	Définit une unité de mesure
<nav>NOUVEAU	Définit un groupe de liens de navigation

<noscript>	Définit une alternative au script non supporté
<object>	Définit un objet du contenu extérieur multimédia
	Définit une liste ordonné
<optgroup>	Regroupe d'une liste d »option dans un formulaire
<option>	Option d'une liste dans un formulaire
<output>NOUVEAU	Définit un type de sortie
<p>	Définit un paragraphe
<param>	Définit les paramètres d'un objet
<pre>	Texte pré-formaté
<progress>NOUVEAU	Définit une progression
<q>	Définit une courte citation
<rp>NOUVEAU	Annotation ruby si le script n'est pas supporté
<rt>NOUVEAU	Annotation ruby d'explication
<ruby>NOUVEAU	Définit une annotation en ruby
<samp>	Définit un échantillon de code
<script>	Définit un script
<section>NOUVEAU	Définit une section
<select>	Définit une liste sélectionnable
<small>	Minimise l'importance d'un texte
<source>NOUVEAU	Définit la source d'un contenu multimédia
	Définit une section de type inline
	Mise en exergue d'un texte – Texte en Gras

<style>	Définit un style CSS
<sub>	Mise en indice d'un texte
<summary>NOUVEAU	Définit l'en-tête des détails d'un document ou section
<sup>	Mise en exposant d'un texte
<svg>NOUVEAU	Définit une image vectorielle
<table>	Définit un tableau
<tbody>	Définit le corps d'un tableau
<td>	Définit une cellule d'un tableau
<textarea>	Définit une zone de texte
<tfoot>	Définit le bas d'un tableau
<th>	Définit une cellule d'en-tête d'un tableau
<thead>	Définit le haut d'un tableau
<time>NOUVEAU	Définit une unité de temps
<title>	Définit le titre d'un document
<tr>	Définit une ligne de tableau
<track>NOUVEAU	Définit une unité de temps pour les éléments <audio> et <video>.
	Définit une liste non-ordonné
<var>	Définit une variable
<video>NOUVEAU	Définit une vidéo
<wbr>NOUVEAU	Définit un possible retour à la ligne

II. Liste des propriétes CSS3 et leurs significationss

http://www.loutor.com/partage/PDF/Liste%20des%20propri_t_s%20CSS%20-%20....pdf

Table des matières

OBJECTIFS DU COURS i

PARTIE I : LE LANGAGE HTML 5 : LES BALISES (Ossature/fond de la page) 1

I. Les balises et les attributs 3

1. Les balises 3
2. Les attributs 4
3. Structure de base d'une page HTML 5 4
4. Les règles de base HTML 5 7
5. L'environnement de travail 9

II. Les différentes balises 10

1. Balises relatives au paragraphe 10
2. Balises relatives au texte 10
3. Balises relatives à la liste 10
4. Balise relative au lien 12
5. Balise relative aux images 15
6. Balise relatives aux audios et aux vidéos 16
7. Balise relative aux figures 17
8. Les balises relatives aux tableaux 18
9. Balises relatives au formulaire 21

PARTIE 2 : LANGAGE CSS 3 - LES STYLES (Ergonomie/forme de la page) 25

I. Les règles générales 25

II. Les sélecteurs 26

1. Sélectionner un seul élément 27
2. Sélectionner plusieurs éléments 27
3. Le sélecteur universel 28

III. Les classes 28

1. Appliquer une classe à un élément 28
2. Appliquer plusieurs classes au même élément 30

IV. Sélecteur d'identifiant id 32

V. Les sélecteurs d'attributs 33

1. Les sélecteurs de valeur d'attribut 34
2. Les sélecteurs de valeur d'attribut CSS 3 35

VI. Les sélecteurs contextuels parent-descendant 36

VII. Les sélecteurs d'éléments adjacents.....38
VIII. La déclaration !important.....39
IX. L'héritage.....40
PARTIE 3 : BOOTSTRAP - Framework CSS.....42
I. Présentation.....42
1. Bibliothèque.....42
2. Découverte de bootstrap.....44
II. Installation de bootstrap.....46
1. Les fichiers de bootstrap.....46
2. Intégration de bootstrap.....48
3. Une trame pour démarrer.....50
4. Flexbox.....54
III. Présentation de la grille.....55
1. Le principe d'une grille.....56
2. Terminologie.....56
3. La grille de Bootstrap.....57
4. Organisation de la grille.....58
5. Le conteneur.....65
6. La grille en action.....68
IV. Mise en page.....75
1. Mise en page simple.....75
2. Mise en page plus chargée.....76
3. Plusieurs asides.....76
V. Mise en forme des tableaux.....78
1. Les tableaux simples en pleine largeur.....78
2. Les tableaux à lignes alternées.....79
3. Les tableaux encadrés.....79
4. Les tableaux avec effet de survol.....80
5. Les tableaux avec effet de survol.....80
VI. Mise en forme des formulaires.....81
1. Structure de base.....81
2. Affichage linéaire du formulaire.....82
3. Affichage Vertical du formulaire.....83

4. Les champs de saisie 84
5. Les boutons 86
VII. Mise en forme de Menu (Horizontal et Vertical) 88
1. La navigation verticale 88
2. La navigation Horizontale 88
VIII. Les composants JavaScript du Bootstrap 89
1. Les fenêtres modales 90
2. Les alertes 91
3. Le Carrousel 91
PARTIE 4 : JAVASCRIPT – PROGRAMMATION (Dynamiser la page) 93
I. Historique du JavaScript 93
II. Les bases du langage JavaScript 93
1. Le concept de variable 93
2. Les types de variables 94
3. Conversion des variables 96
4. Les opérateurs 97
III. Le DOM HTML (Document Object Model) 99
1. Qu'est-ce que le DOM HTML ? 100
2. Méthodes HTML DOM 101
3. La méthode getElementById 101
4. La propriété innerHTML 102
5. Recherche d'éléments HTML 102
6. Modification des éléments HTML 102
7. Ajout et suppression d'éléments 103
8. Ajout de gestionnaires d'événements 103
9. Quelques possibilités d'affichage JavaScript 103
IV. Les boites de dialogues (alert, confirm, prompt) 106
1. Alert() 107
2. Confirm() 107
3. Prompt() 109
V. Les structures conditionnelles/alternatives (IF..ELSE, SWITCH) 110
1. Structure IF 110

2. Structure SWITCH 110
VI. Les structures itératives (WHILE, DO…WHILE, FOR, FOR…IN) 110
1. La structure while 111
2. La structure Do .. while 111
3. La structure For 111
4. La structure For .. In 112
VII. Les tableaux 112
1. Tableau (type élémentaire) 112
2. Quelques méthodes tableau 112
VIII. Les fonctions 113
1. Déclaration de fonction 113
2. Expressions de fonction 114
IX. Les évènements 114
ANNEXE 118
I. La liste complète des balises HTML5 118
II. Liste des propriétes CSS3 et leurs significationss 122